Java cheatsheet · 19 chapters · only Java code

Playwright with Java: cheatsheet

The key ideas, every example program in Java, the tables and the common mistakes from every chapter, in one place for quick revision. Everything here comes from the chapters and shows Java only, so this sheet is all you need. Each card links back to the lesson that teaches it.

95 key ideas | 97 code examples | 77 tables | 72 tips and traps

STEP 1 Getting started

1.1 What Playwright is and how it works

What Playwright is and the jobs teams use it for, the official languages, a fact based comparison with Selenium and Cypress, a first script in four languages, how one call travels through the client library, the driver and the browser protocol with real DEBUG=pw:api, pw:channel and pw:protocol logs, a first test with JUnit, pytest and Playwright Test, a recorded sign in walk, and what a test runner does for every test.

KEY IDEAS

- **You write steps** Open, type, click, check. In Java, Python, JavaScript or TypeScript.
- **Messages** Each call becomes a small message. The driver is the same for every language.
- **Real browsers** Real clicks and real typing. The page cannot tell it apart from a person.
- **It waits** Visible, stable, enabled. No fixed sleeps, so far fewer flaky tests.
- **Remember** Playwright drives real browsers through one driver, waits for elements before it acts, and works the same way in Java, Python, JavaScript and TypeScript.

CODE

Your first Playwright script

src/main/java/com/brainy/FirstScript.java

```
package com.brainy;

import com.microsoft.playwright.*;

public class FirstScript {
  public static void main(String[] args) {
    try (Playwright playwright = Playwright.create()) {        // start the Playwright driver
      Browser browser = playwright.chromium().launch();         // start a real Chromium, with no window
      Page page = browser.newPage();                              // open a new tab
      page.navigate("http://localhost:4173/products");
      System.out.println("Title: " + page.title());
      page.screenshot(new Page.ScreenshotOptions()
          .setPath(java.nio.file.Paths.get("products.png")));  // save a picture of the page
    }                                                               // closing Playwright closes the browser
too
  }
}
```

Run it: `mvn -q compile exec:java`

Lesson: Your first script

Watch what Playwright does inside each call

src/main/java/com/brainy/FirstScript.java

```
package com.brainy;

import com.microsoft.playwright.*;
import com.microsoft.playwright.options.AriaRole;

public class FirstScript {
  public static void main(String[] args) {
```

```
    try (Playwright playwright = Playwright.create()) {
      Browser browser = playwright.chromium().launch();
      Page page = browser.newPage();
      page.navigate("http://localhost:4173/login");
      page.getByLabel("Email").fill("asha@shop.example");
      page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Sign in")).click();
    }
  }
}
```

Run it: `DEBUG=pw:api mvn -q compile exec:java`

Lesson: How one line travels to the browser

Your first test, in four languages

src/test/java/com/brainy/SignInTest.java

```
package com.brainy;

import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class SignInTest {
  @Test
  void aCustomerCanSignIn(Page page) {
    page.navigate("/login");
    page.getByLabel("Email").fill("asha@shop.example");
    page.getByLabel("Password").fill("s3cret");
    page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Sign in")).click();
    assertThat(page.getByText("Hello, Asha")).isVisible();
  }
}
```

Run it: `mvn test`

Lesson: From a script to a test

A customer signs in

`page.navigate("http://localhost:4173/login");`	Open the sign in page.
`page.getByLabel("Email").fill("asha@shop.example");`	Type the email into the box labelled Email.
`page.getByLabel("Password").fill("s3cret");`	Type the password.
`page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Sign in")).click();`	Click the button called Sign in. The shop signs Asha in and opens the products page.
`assertThat(page.getByText("Hello, Asha")).isVisible();`	Check that the greeting says Hello, Asha. The check waits until it appears.

Lesson: From a script to a test

TABLES

Table: language · package · usual test runner · version used on this page

Language	Package	Usual test runner	Version used on this page
☕ Java	`com.microsoft.playwright:playwright` on Maven Central	JUnit 5 or TestNG	1.63.0

Lesson: What Playwright is

Table: playwright · selenium webdriver · cypress

	Playwright	Selenium WebDriver	Cypress
Languages	Java, Python, JavaScript, TypeScript, C#	Java, Python, JavaScript, C#, Ruby	JavaScript and TypeScript
Browsers	Chromium, Firefox, WebKit	Chrome, Firefox, Edge, Safari through their drivers	Chromium family, Firefox, WebKit (experimental)
How it talks to the browser	Browser debugging protocols, through one driver process	The W3C WebDriver protocol, through a driver per browser	Runs inside the browser next to your app
Waiting	Automatic before every action and in every check	You add waits yourself	Automatic retries for commands and checks
Several tabs, windows and domains in one test	Yes	Yes	Limited
Parallel runs	Built in, free	With Selenium Grid or a test runner	Paid cloud service, or plugins
Debugging	Trace viewer, UI mode, inspector, codegen	Logs and screenshots you add	Time travel in the runner

Lesson: Why teams pick Playwright

Table: job · ☕ java

Job	☕ Java
Run the tests	`mvn test`
Where settings live	an options class, or your own base class
A fresh page for each test	`@UsePlaywright`, or your own setup methods
Checks that wait	`assertThat(...)`
Run in parallel	JUnit or TestNG parallel settings
HTML report	Allure or ExtentReports

Lesson: What the test runner does for you

Table: module · you will learn

Module	You will learn
🚀 Getting started	what Playwright is, how it works inside, installing it in four languages, codegen and VS Code
🌐 Browser handling	browsers, contexts, pages, devices, tabs, pop ups, frames, dialogs, downloads and uploads
🎯 Finding and using elements	locators, actions and auto waiting, and checks that wait
🏃 The test runner	configuration, tags, data driven tests, fixtures, hooks, parallel runs, retries and timeouts
🔌 Network and API	mocking the server, recording traffic, API tests and signing in once
🐞 Debugging and quality	trace viewer, UI mode, screenshots, videos, visual and accessibility tests, the clock
🏗️ Framework design	page objects, a real company framework built step by step in each language, CI and Docker
📊 Reporting labs	the built in reporters, HTML report, Allure, pytest-html, ExtentReports and your own reporter
🆕 New and AI features	what changed from version 1.45 to 1.64, the Playwright MCP server and test agents

Lesson: What this subject covers

TIPS AND TRAPS

COMMON MISTAKE People often ask for Playwright in C, C++ or Go. There is no official Playwright for C or C++. For Go there is a community project, playwright-go, which is not made by the Playwright team. This subject sticks to the official languages so that every example is supported and up to date.

Lesson: What Playwright is

IN REAL TEAMS In many companies you will join a project that already uses Selenium. Knowing both is useful. Many teams move to Playwright one test suite at a time, because the ideas are close: find an element, act on it, check the result.

Lesson: Why teams pick Playwright

REMEMBER The driver is the same Node.js program for every language. The Java jar and the Python package carry a copy of it and start it for you. This is why a new feature arrives in Java, Python, JavaScript and C# in the same release.

Lesson: How one line travels to the browser

REMEMBER The test method receives a `Page` because of `@UsePlaywright(ShopOptions.class)`. JUnit creates a fresh context and page for each test. ShopOptions sets the base address, so `navigate("/login")` means http://localhost:4173/login.

Lesson: From a script to a test

1.2 Installing Playwright and your first project

Runtime and operating system requirements, installing with Maven, pip and npm init playwright, checking the version, where browsers are installed and Chrome for Testing, the real files of a scaffolded project and typical Python and Java layouts, codegen with real screenshots and the recorded code in four languages, cleaning up recorded code, running, filtering and listing tests, editors and updating safely.

KEY IDEAS

- **Runtime** Install your language first. Check it with a version command.
- **Package** Maven, pip or npm. Pin the exact version.
- **Browsers** Playwright brings its own. --with-deps adds the Linux libraries.
- **Record** Click around, get code. Then clean it up and add a check.
- **Remember** Install the runtime, the Playwright package and its browsers, record a first draft with codegen, then turn it into a real test with a clear name and a check.

CODE

Check what is installed

pom.xml

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.brainy</groupId>
  <artifactId>shop-tests</artifactId>
  <version>1.0.0</version>
  <properties>
    <maven.compiler.release>21</maven.compiler.release>
  </properties>
  <dependencies>
    <dependency>
      <groupId>com.microsoft.playwright</groupId>
      <artifactId>playwright</artifactId>
      <version>1.63.0</version>
    </dependency>
  </dependencies>
  <build>
    <plugins>
      <plugin>
        <groupId>org.codehaus.mojo</groupId>
        <artifactId>exec-maven-plugin</artifactId>
        <version>3.5.1</version>
      </plugin>
    </plugins>
  </build>
</project>
```

Run it: `mvn -q exec:java -D exec.mainClass=com.microsoft.playwright.CLI -D exec.args="--version"`

Lesson: Install Playwright in your language

Where the browsers live

Run it: `mvn -q exec:java -D exec.mainClass=com.microsoft.playwright.CLI -D exec.args="install --dry-run chromium"`

Lesson: Install Playwright in your language

What codegen recorded, run as it is

src/test/java/TestExample.java

```
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.options.*;

import org.junit.jupiter.api.*;
import static com.microsoft.playwright.assertions.PlaywrightAssertions.*;

@UsePlaywright
public class TestExample {
  @Test
  void test(Page page) {
    page.navigate("http://localhost:4173/login");
    page.getByRole(AriaRole.TEXTBOX, new Page.GetByRoleOptions().setName("Email")).click();
    page.getByRole(AriaRole.TEXTBOX, new
Page.GetByRoleOptions().setName("Email")).fill("asha@shop.example");
    page.getByRole(AriaRole.TEXTBOX, new Page.GetByRoleOptions().setName("Password")).click();
    page.getByRole(AriaRole.TEXTBOX, new Page.GetByRoleOptions().setName("Password")).fill("s3cret");
    page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Sign in")).click();
  }
}
```

Run it: `mvn test`

Lesson: Record a test with codegen

The same test after a clean up

src/test/java/com/brainy/SignInTest.java

```
package com.brainy;

import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class SignInTest {
  @Test
  void aCustomerCanSignIn(Page page) {
    page.navigate("/login");
    page.getByRole(AriaRole.TEXTBOX, new
Page.GetByRoleOptions().setName("Email")).fill("asha@shop.example");
    page.getByRole(AriaRole.TEXTBOX, new Page.GetByRoleOptions().setName("Password")).fill("s3cret");
    page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Sign in")).click();
    assertThat(page.getByText("Hello, Asha")).isVisible();
  }
}
```

Run it: `mvn test`

Lesson: Record a test with codegen

Run only one test from a file

src/test/java/com/brainy/ShopTest.java

```
package com.brainy;

import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;
```

```java
import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class ShopTest {
  @Test
  void aCustomerCanSignIn(Page page) {
    page.navigate("/login");
    page.getByLabel("Email").fill("asha@shop.example");
    page.getByLabel("Password").fill("s3cret");
    page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Sign in")).click();
    assertThat(page.getByText("Hello, Asha")).isVisible();
  }

  @Test
  void theProductsPageHasATitle(Page page) {
    page.navigate("/products");
    assertThat(page).hasTitle("Products · Brainy Shop");
  }
}
```

Run it: `mvn test -Dtest="ShopTest#aCustomerCanSignIn"`

Lesson: Running tests from the terminal

List the tests without running them

src/test/java/com/brainy/ShopTest.java

```java
package com.brainy;

import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class ShopTest {
  @Test
  void aCustomerCanSignIn(Page page) {
    page.navigate("/login");
    page.getByLabel("Email").fill("asha@shop.example");
    page.getByLabel("Password").fill("s3cret");
    page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Sign in")).click();
    assertThat(page.getByText("Hello, Asha")).isVisible();
  }

  @Test
  void theProductsPageHasATitle(Page page) {
    page.navigate("/products");
    assertThat(page).hasTitle("Products · Brainy Shop");
  }
}
```

Run it: `mvn -q test-compile && find target/test-classes -name "*Test.class"`

Lesson: Running tests from the terminal

TABLES

Table: ☕ java

	☕ Java
Runtime	Java 8 or later (this page uses Java 21)
Package tool	Maven or Gradle
Editor most teams use	IntelliJ IDEA
Operating system	Windows 11, macOS 14 or later, Debian 12 or 13, Ubuntu 22.04, 24.04 or 26.04

Lesson: What you need before you start

Table: i want to · ☕ java (maven)

I want to	☕ Java (Maven)
run every test	`mvn test`
run one file	`mvn test -Dtest=ShopTest`
run tests by name	`-Dtest="ShopTest#signIn"`
see the browser	`setHeadless(false)` in the options
slow every action down	`setSlowMo(500)` in the launch options
pick a browser	`setBrowserName("firefox")`
debug step by step	`PWDEBUG=1 mvn test`
run 4 at a time	JUnit parallel settings

Lesson: Running tests from the terminal

Table: language · update the package · then

Language	Update the package	Then
☕ Java	change `<version>` in pom.xml	run the tests. The new browsers download on first use, or run the CLI `install` command

Lesson: Updating Playwright safely

TIPS AND TRAPS

COMMON MISTAKE Keep one Playwright version per project and update on purpose. Each Playwright release expects its own browser builds. After you update the package, run the install command again, or the tests stop with an error that says the browser executable does not exist.

Lesson: Install Playwright in your language

IN REAL TEAMS Codegen picks locators the same way you should: role and accessible name first, then label, placeholder or text, and a test id only when nothing better exists. When the recorded locator looks odd, the page itself is often missing a label, which is worth reporting to the developers.

Lesson: Record a test with codegen

IN REAL TEAMS Turn on format on save and a linter in your editor from day one.

Lesson: Your editor

REMEMBER Pin exact versions in your project (no ^ in package.json, == in requirements.txt). Then a teammate and the CI server always run the same Playwright, and an update is a change you can see and review.

Lesson: Updating Playwright safely

STEP 2 Browser handling

2.1 Browsers, contexts, pages and devices

The browser, context and page levels with an interactive builder, launch options such as headless, slowMo and channel, two signed in users in one browser, context options for viewport, locale, time zone, colour scheme, touch, geolocation and permissions, device profiles, new tabs and popups, Chromium, Firefox and WebKit projects, persistent profiles, connectOverCDP and browser servers.

KEY IDEAS

- **Browser** Start once, about 0.5 s. Shared by many tests.
- **Context** A fresh, empty profile. Its own cookies, storage and permissions.

- **Page** One tab. Tabs in a context share the login.
- **Emulation** Size, locale, time zone, location. Set on the context, nothing on your computer changes.
- **Remember** Start one browser, give every test a new context, and set what the site should see, such as a phone or a German user, with context options.

CODE

Launch a browser with options

src/main/java/com/brainy/Main.java

```
package com.brainy;

import com.microsoft.playwright.*;
import java.util.List;

public class Main {
  public static void main(String[] args) {
    try (Playwright playwright = Playwright.create()) {
      Browser browser = playwright.chromium().launch(new BrowserType.LaunchOptions()
          .setHeadless(true)          // no window. Use false to watch it work
          .setSlowMo(100)             // wait 100 ms before every action
          .setArgs(List.of("--lang=en-GB")));
      System.out.println("Browser type: " + browser.browserType().name());
      System.out.println("Version: " + browser.version());
      System.out.println("Connected: " + browser.isConnected());
      System.out.println("Contexts so far: " + browser.contexts().size());
      browser.close();
      System.out.println("Connected after close: " + browser.isConnected());
    }
  }
}
```

Run it: `mvn -q compile exec:java`

Lesson: Launching a browser

Two shoppers in one browser, kept apart

src/test/java/com/brainy/ContextTest.java

```
package com.brainy;

import com.microsoft.playwright.*;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class ContextTest {
  @Test
  void twoShoppersInOneBrowser(Browser browser) {
    // Each context is a separate, empty browser profile.
    BrowserContext ashaContext = browser.newContext();
    BrowserContext guestContext = browser.newContext();
    Page asha = ashaContext.newPage();
    Page guest = guestContext.newPage();

    asha.navigate("http://localhost:4173/login");
    asha.getByLabel("Email").fill("asha@shop.example");
    asha.getByLabel("Password").fill("s3cret");
    asha.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Sign in")).click();
    assertThat(asha.getByText("Hello, Asha")).isVisible();

    guest.navigate("http://localhost:4173/products");
    assertThat(guest.getByRole(AriaRole.LINK, new Page.GetByRoleOptions().setName("Sign in"))).isVisible();
    System.out.println("Asha cookies: " + ashaContext.cookies().size());
    System.out.println("Guest cookies: " + guestContext.cookies().size());

    ashaContext.close();
    guestContext.close();
```

```
  }
}
```

Run it: `mvn test`

Lesson: Contexts keep tests apart

Pretend to be a German phone user in India

src/test/java/com/brainy/EmulationTest.java

```
package com.brainy;

import com.microsoft.playwright.*;
import com.microsoft.playwright.junit.Options;
import com.microsoft.playwright.junit.OptionsFactory;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.ColorScheme;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(EmulationTest.GermanPhone.class)
public class EmulationTest {
  public static class GermanPhone implements OptionsFactory {
    @Override
    public Options getOptions() {
      return new Options().setHeadless(true).setBaseUrl("http://localhost:4173")
          .setContextOptions(new Browser.NewContextOptions()
              .setViewportSize(390, 844)
              .setLocale("de-DE")
              .setTimezoneId("Asia/Kolkata")
              .setColorScheme(ColorScheme.DARK)
              .setHasTouch(true));
    }
  }

  @Test
  void theShopSeesAGermanPhoneUserInIndia(Page page) {
    page.navigate("/about-you");
    for (String id : new String[] {"width", "lang", "tz", "price", "touch"}) {
      System.out.println(String.format("%-6s ", id) + page.locator("#" + id).textContent());
    }
    Object dark = page.evaluate("matchMedia('(prefers-color-scheme: dark)').matches");
    System.out.println("dark   " + dark);
    assertThat(page.locator("#lang")).hasText("de-DE");
  }
}
```

Run it: `mvn test`

Lesson: Pretend to be another user

Location and permissions

src/test/java/com/brainy/LocationTest.java

```
package com.brainy;

import com.microsoft.playwright.*;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class LocationTest {
  @Test
  void findsStoresNearNewDelhi(Browser browser) {
    BrowserContext context = browser.newContext(new Browser.NewContextOptions()
        .setGeolocation(28.6139, 77.209)                // New Delhi
        .setPermissions(java.util.List.of("geolocation")));
    Page page = context.newPage();
    page.navigate("http://localhost:4173/about-you");
    page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Find stores near me")).click();
    assertThat(page.locator("#where")).hasText("28.6139, 77.2090");
```

```
    context.close();
  }

  @Test
  void locationBlockedWhenPermissionIsNotGranted(Page page) {
    page.navigate("/about-you");
    page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Find stores near me")).click();
    assertThat(page.locator("#where")).containsText("blocked");
  }
}
```

Run it: `mvn test`

Lesson: Pretend to be another user

Emulate a phone from the device list

src/test/java/com/brainy/DeviceTest.java

```
package com.brainy;

import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.Options;
import com.microsoft.playwright.junit.OptionsFactory;
import com.microsoft.playwright.junit.UsePlaywright;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(DeviceTest.Pixel.class)
public class DeviceTest {
  public static class Pixel implements OptionsFactory {
    @Override
    public Options getOptions() {
      return new Options().setHeadless(true).setBaseUrl("http://localhost:4173").setDeviceName("Pixel 7");
    }
  }

  @Test
  void theShopOnAPixel7(Page page) {
    page.navigate("/about-you");
    System.out.println("width  " + page.locator("#width").textContent());
    System.out.println("touch  " + page.locator("#touch").textContent());
    System.out.println("mobile " + page.evaluate("/Mobile/.test(navigator.userAgent)"));
    assertThat(page.locator("#touch")).hasText("yes");
  }
}
```

Run it: `mvn test`

Lesson: Phones and tablets

New tabs and pop up windows

src/test/java/com/brainy/TabsTest.java

```
package com.brainy;

import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class TabsTest {
  @Test
  void helpOpensInANewTab(Page page) {
    page.navigate("/widgets");
    // waitForPopup runs the click and waits for the new tab it opens.
    Page help = page.waitForPopup(() ->
        page.getByRole(AriaRole.LINK, new Page.GetByRoleOptions().setName("Open help in a new
tab")).click());

    assertThat(help).hasTitle("Help · Brainy Shop");
    System.out.println("Tabs open: " + page.context().pages().size());
```

```
    help.close();
    System.out.println("Tabs open: " + page.context().pages().size());
  }

  @Test
  void theChatWindowIsAPopup(Page page) {
    page.navigate("/widgets");
    Page chat = page.waitForPopup(() ->
        page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Open chat window")).click());
    assertThat(chat.getByRole(AriaRole.HEADING)).hasText("Chat with us");
  }
}
```

Run it: `mvn test`

Lesson: New tabs and pop ups

A persistent profile that survives a restart

src/main/java/com/brainy/Main.java

```
package com.brainy;

import com.microsoft.playwright.*;
import java.nio.file.Files;
import java.nio.file.Path;

public class Main {
  public static void main(String[] args) throws Exception {
    Path profile = Files.createTempDirectory("shop-profile-");
    try (Playwright playwright = Playwright.create()) {
      // First visit: a real profile folder on disk, like a person's Chrome profile.
      BrowserContext context = playwright.chromium().launchPersistentContext(profile, new
BrowserType.LaunchPersistentContextOptions());
      Page page = context.pages().get(0);
      page.navigate("http://localhost:4173/");
      page.evaluate("localStorage.setItem('theme', 'dark')");
      context.close();

      // Second visit with the same folder: the data is still there.
      context = playwright.chromium().launchPersistentContext(profile, new
BrowserType.LaunchPersistentContextOptions());
      page = context.pages().get(0);
      page.navigate("http://localhost:4173/");
      System.out.println("theme = " + page.evaluate("localStorage.getItem('theme')"));
      context.close();
    }
  }
}
```

Run it: `mvn -q compile exec:java`

Lesson: Profiles, closing and remote browsers

TABLES

Table: option · what it does · when you use it

Option	What it does	When you use it
headless	run without a window (the default)	always on CI, turn off to watch a test locally
slowMo	pause before each action	to follow a test with your eyes or record a demo
channel	use an installed branded browser: `chrome`, `msedge`, `chrome-beta`	when the team must test the exact browser users have
args	extra command line switches for the browser	rare cases, such as a language or a fake camera
executablePath	start a browser from a path you give	company images with their own browser build
timeout	how long to wait for the browser to start	slow CI machines

Lesson: Launching a browser

Table: option · example · what the page sees

Option	Example	What the page sees
viewport	390 × 844	`innerWidth`, media queries
locale	de-DE	`navigator.language`, number and date formats
timezoneId	Asia/Kolkata	dates and times
colorScheme	dark	`prefers-color-scheme`
hasTouch, isMobile	true	touch events, mobile layout
geolocation, permissions	New Delhi	the location API
userAgent	any string	`navigator.userAgent`
offline	true	no network at all

Lesson: Pretend to be another user

Table: ☕ java

	☕ Java
pick the engine	`playwright.firefox()` or `setBrowserName("firefox")`
all three in one run	a parameterized test or three Maven runs
branded Chrome or Edge	`setChannel("chrome")`

Lesson: Chromium, Firefox and WebKit

TIPS AND TRAPS

IN REAL TEAMS Real projects use two contexts in one test for features with two people: a chat between a customer and a support agent, an admin approving an order while the customer waits, or two players in a game. Each person gets their own context.

Lesson: Contexts keep tests apart

COMMON MISTAKE Emulation catches most layout and touch bugs and costs nothing. Some bugs only show on real hardware, such as a keyboard covering a field or a slow phone. Many teams run emulated phones on every commit and real devices in a cloud service before a release.

Lesson: Phones and tablets

REMEMBER WebKit on Windows and Linux is the engine of Safari, not Safari itself. It catches most Safari bugs. For the last few percent, teams run a small smoke suite on real Safari on a Mac.

Lesson: Chromium, Firefox and WebKit

REMEMBER Close what you open. In a test runner the fixtures close contexts and pages for you. In a script, close the browser at the end, or use try with resources in Java and a with block in Python, so a crash does not leave browser processes running on the machine.

Lesson: Profiles, closing and remote browsers

2.2 Navigation, frames, dialogs, files and events

What goto waits for, waiting for the URL after a click, back, forward and reload, a recorded walk through a payment iframe, frameLocator and contentFrame, alert, confirm and prompt with the default dismiss, downloads with suggestedFilename and saveAs, uploads from memory, console, pageerror and request events, evaluate and addInitScript, and locating inside shadow DOM.

KEY IDEAS

- **Navigation** goto waits for load. After a click, wait for the new URL.
- **Iframes** Step in first. Then locate as usual.

- **Dialogs** Dismissed by default. Handle them before the click.
- **Files** Wait for downloads. Fill file inputs directly.
- **Remember** Start waiting before the click for anything the click creates: a new URL, a popup, a dialog or a download.

CODE

Go, wait for the address, go back and reload

src/test/java/com/brainy/NavigationTest.java

```java
package com.brainy;

import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import com.microsoft.playwright.options.WaitUntilState;
import org.junit.jupiter.api.Test;
import java.util.regex.Pattern;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class NavigationTest {
  @Test
  void movingAroundTheShop(Page page) {
    page.navigate("/login", new Page.NavigateOptions().setWaitUntil(WaitUntilState.DOMCONTENTLOADED));
    System.out.println("1. " + page.url());

    page.getByLabel("Email").fill("asha@shop.example");
    page.getByLabel("Password").fill("s3cret");
    page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Sign in")).click();
    page.waitForURL("**/products");                  // the app moves on by itself
    System.out.println("2. " + page.url());

    page.goBack();
    assertThat(page).hasURL(Pattern.compile("/login$"));
    System.out.println("3. " + page.url());

    page.goForward();
    page.reload();
    assertThat(page).hasURL(Pattern.compile("/products$"));
    System.out.println("4. " + page.url() + " | " + page.title());
  }
}
```

Run it: `mvn test`

Lesson: Going to pages and waiting for them

Paying inside an iframe

`page.navigate("http://localhost:4173/frames");`	Open the checkout page. The card form is an iframe from the payment company.
`page.frameLocator("iframe[title=\"Payment form\"]").getByLabel("Card number").fill("4111 1111 1111 1111");`	Step into the iframe and type the card number.
`page.frameLocator("iframe[title=\"Payment form\"]").getByLabel("Expiry").fill("12/29");`	Type the expiry date.
`page.frameLocator("iframe[title=\"Payment form\"]").getByRole(AriaRole.BUTTON, new FrameLocator.GetByRoleOptions().setName("Pay ₹219")).click();`	Click Pay inside the iframe.
`assertThat(page.getByRole(AriaRole.STATUS)).hasText("Paid with card ending 1111");`	Back in the main page, the shop shows the result.

Lesson: Iframes

Fill a card form that lives inside an iframe

src/test/java/com/brainy/FrameTest.java

```java
package com.brainy;

import com.microsoft.playwright.FrameLocator;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class FrameTest {
  @Test
  void payInsideThePaymentIframe(Page page) {
    page.navigate("/frames");
    FrameLocator payment = page.frameLocator("iframe[title=\"Payment form\"]");

    payment.getByLabel("Card number").fill("4111 1111 1111 1111");
    payment.getByLabel("Expiry").fill("12/29");
    payment.getByRole(AriaRole.BUTTON, new FrameLocator.GetByRoleOptions().setName("Pay ₹219")).click();

    // The message appears in the main page, outside the iframe.
    assertThat(page.getByRole(AriaRole.STATUS)).hasText("Paid with card ending 1111");
    System.out.println("Frames on the page: " + page.frames().size());
  }

  @Test
  void aShortCardNumberIsRefused(Page page) {
    page.navigate("/frames");
    // Another way in: find the iframe element, then step into it.
    FrameLocator payment = page.getByTitle("Payment form").contentFrame();
    payment.getByLabel("Card number").fill("4111");
    payment.getByRole(AriaRole.BUTTON, new FrameLocator.GetByRoleOptions().setName("Pay ₹219")).click();
    assertThat(payment.getByRole(AriaRole.ALERT)).hasText("Card number must have 16 digits");
  }
}
```

Run it: `mvn test`

Lesson: Iframes

Alerts, confirms and prompts

src/test/java/com/brainy/DialogTest.java

```java
package com.brainy;

import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class DialogTest {
  @Test
  void withNoHandlerDialogsAreDismissed(Page page) {
    page.navigate("/widgets");
    page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Delete account")).click();
    assertThat(page.getByRole(AriaRole.STATUS)).hasText("Kept your account");
  }

  @Test
  void acceptAConfirmAndReadItsMessage(Page page) {
    page.navigate("/widgets");
    page.onDialog(dialog -> {
      System.out.println(dialog.type() + " | " + dialog.message());
      dialog.accept();
    });
    page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Delete account")).click();
```

```
    assertThat(page.getByRole(AriaRole.STATUS)).hasText("Account deleted");
  }

  @Test
  void typeAnAnswerIntoAPrompt(Page page) {
    page.navigate("/widgets");
    page.onceDialog(dialog -> dialog.accept("Weekly shop"));
    page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Rename list")).click();
    assertThat(page.getByRole(AriaRole.STATUS)).hasText("List renamed to Weekly shop");
  }
}
```

Run it: `mvn test`

Lesson: Alerts, confirms and prompts

Catch a download and read the file

src/test/java/com/brainy/DownloadTest.java

```
package com.brainy;

import com.microsoft.playwright.Download;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;
import java.nio.file.Files;
import java.nio.file.Paths;

import static org.junit.jupiter.api.Assertions.assertEquals;

@UsePlaywright(ShopOptions.class)
public class DownloadTest {
  @Test
  void downloadTheOrdersReport(Page page) throws Exception {
    page.navigate("/widgets");
    Download download = page.waitForDownload(() ->
        page.getByRole(AriaRole.LINK, new Page.GetByRoleOptions().setName("Download report")).click());

    System.out.println("File name: " + download.suggestedFilename());
    String text = Files.readString(download.path());
    System.out.println(text.trim());
    assertEquals("order,total", text.split("\n")[0]);
    download.saveAs(Paths.get("reports", download.suggestedFilename()));   // keep a copy
  }
}
```

Run it: `mvn test`

Lesson: Downloads and uploads

Upload files without touching the disk

src/test/java/com/brainy/UploadTest.java

```
package com.brainy;

import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import com.microsoft.playwright.options.FilePayload;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class UploadTest {
  @Test
  void sendFeedbackWithTwoScreenshots(Page page) {
    page.navigate("/forms");
    page.getByPlaceholder("Full name").fill("Asha");
    page.getByLabel("Screenshots").setInputFiles(new FilePayload[] {
        new FilePayload("cart.png", "image/png", "fake picture 1".getBytes()),
        new FilePayload("error.png", "image/png", "fake picture 2".getBytes())});
    page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Send feedback")).click();
    assertThat(page.getByRole(AriaRole.STATUS)).containsText("files: cart.png, error.png");
```

```
  }
}
```

Run it: `mvn test`

Lesson: Downloads and uploads

Listen to console messages, page errors and requests

src/test/java/com/brainy/EventsTest.java

```
package com.brainy;

import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;
import java.util.ArrayList;
import java.util.List;

@UsePlaywright(ShopOptions.class)
public class EventsTest {
  @Test
  void catchConsoleMessagesAndPageErrors(Page page) {
    List<String> messages = new ArrayList<>();
    List<String> errors = new ArrayList<>();
    page.onConsoleMessage(msg -> messages.add(msg.type() + ": " + msg.text()));
    page.onPageError(errors::add);
    page.onRequest(request -> {
      if (request.url().contains("/api/")) System.out.println("request -> " + request.method() + " " +
request.url());
    });
    page.navigate("/widgets");

    page.waitForConsoleMessage(new Page.WaitForConsoleMessageOptions().setPredicate(m ->
m.type().equals("warning")),
        () -> page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Write to the
console")).click());
    System.out.println(messages);

    page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Break the page")).click();
    page.waitForCondition(() -> !errors.isEmpty());
    System.out.println("Page error: " + errors.get(0));
  }
}
```

Run it: `mvn test`

Lesson: Listening to what the page does

Run JavaScript inside the page

src/test/java/com/brainy/EvaluateTest.java

```
package com.brainy;

import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;
import java.util.List;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class EvaluateTest {
  @Test
  void runCodeInsideThePage(Page page) {
    // Runs in the page before any of its own scripts, on every navigation.
    page.addInitScript("window.testMode = true");
    page.navigate("/products");
    assertThat(page.getByRole(AriaRole.LISTITEM)).hasCount(5);   // wait for the list

    Object info = page.evaluate("() => ({" +
        " title: document.title," +
        " cards: document.querySelectorAll('li.card').length," +
        " testMode: window.testMode })");
```

```
    System.out.println(info);

    Object sum = page.evaluate("([a, b]) => a + b", List.of(20, 199));   // pass values in
    System.out.println("sum = " + sum);

    Object tag = page.getByPlaceholder("Search products").evaluate("el => el.tagName");
    System.out.println("tag = " + tag);
  }
}
```

Run it: `mvn test`

Lesson: Running code inside the page

Click a button inside a shadow DOM component

src/test/java/com/brainy/ShadowTest.java

```
package com.brainy;

import com.microsoft.playwright.Locator;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class ShadowTest {
  @Test
  void voteInsideAShadowDomComponent(Page page) {
    page.navigate("/shadow");
    Locator ashaReview = page.locator("review-card").filter(new Locator.FilterOptions().setHasText("Asha"));

    // getByRole and CSS look inside open shadow roots by themselves.
    ashaReview.getByRole(AriaRole.BUTTON, new Locator.GetByRoleOptions().setName("Helpful")).click();
    assertThat(ashaReview.getByText("found this helpful")).hasText("1 person found this helpful");

    System.out.println("CSS sees the buttons: " + page.locator("review-card button").count());
    System.out.println("XPath sees the buttons: " + page.locator("//review-card//button").count());
  }
}
```

Run it: `mvn test`

Lesson: Shadow DOM and web components

TABLES

Table: dialog · opened by · accept() · dismiss()

Dialog	Opened by	accept()	dismiss()
alert	`alert(msg)`	closes it	closes it
confirm	`confirm(msg)`	the page gets true	the page gets false
prompt	`prompt(msg, default)`	the page gets your text	the page gets null
beforeunload	leaving a page with unsaved changes	leave the page	stay on the page

Lesson: Alerts, confirms and prompts

Table: event · tells you · typical use

Event	Tells you	Typical use
console	a message the page logged	fail on `console.error`, debug a test
pageerror	an error nothing caught	fail the test on broken JavaScript
request and response	every network call	check an API was called, count calls
popup, download, dialog	a new tab, a file, a dialog	handle it in the test
close, crash	the page closed or crashed	clean up, report a crash

Lesson: Listening to what the page does

TIPS AND TRAPS

COMMON MISTAKE Do not wait for the network to go quiet. A chat widget or analytics script can keep it busy forever. Wait for what the user is waiting for: a URL, a heading or a message. Every check with expect already waits.

Lesson: Going to pages and waiting for them

REMEMBER Some pages hide the real file input and open the file window from a pretty button. Then wait for the file chooser event: `page.waitForFileChooser(...)` in Java, and call `setFiles` on it.

Lesson: Downloads and uploads

IN REAL TEAMS Many companies add one shared fixture that collects page errors and fails any test that had one.

Lesson: Listening to what the page does

STEP 3 Finding and using elements

3.1 Locators: finding elements

Why a locator is searched again every time, a live locator lab, which locator to try first, getByRole with names, levels and states, labels, placeholders, text, titles and test ids, custom test id attributes, CSS and XPath and why they break, filter, chaining, nth, and, or, strict mode violations, and a recorded walk through a sortable, paged table.

KEY IDEAS

- **Role and name** What the user sees. The first choice.
- **Forms** Boxes by their label. Placeholder when there is none.
- **Narrow down** Card first, then button. hasText, has, hasNot.
- **One element** Actions need exactly one. The error lists them all.
- **Remember** A locator is searched again every time you use it, and an action needs exactly one match.

CODE

A locator is found again every time you use it

src/test/java/com/brainy/LazyTest.java

```
package com.brainy;

import com.microsoft.playwright.Locator;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class LazyTest {
  @Test
```

```
  void aLocatorIsARecipeNotAnElement(Page page) {
    page.navigate("/products");
    Locator cards = page.getByRole(AriaRole.LISTITEM);          // nothing is searched yet
    assertThat(cards).hasCount(5);
    System.out.println("before search: " + cards.count());

    page.getByPlaceholder("Search products").fill("o");
    System.out.println("after \"o\": " + cards.count());       // the same locator, found again

    page.getByPlaceholder("Search products").fill("bottle");
    assertThat(cards).hasCount(1);
    System.out.println("after \"bottle\": " + cards.count());
    System.out.println(cards.describe("product cards"));
  }
}
```

Run it: `mvn test`

Lesson: What a locator is

getByRole with names, levels and states

src/test/java/com/brainy/RoleTest.java

```
package com.brainy;

import com.microsoft.playwright.Locator;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class RoleTest {
  @Test
  void findThingsTheWayAUserSeesThem(Page page) {
    page.navigate("/products");
    assertThat(page.getByRole(AriaRole.LISTITEM)).hasCount(5);   // waits for the list

    System.out.println("headings: " + page.getByRole(AriaRole.HEADING).allTextContents());
    System.out.println("level 1 heading: " + page.getByRole(AriaRole.HEADING, new
Page.GetByRoleOptions().setLevel(1)).textContent());

    // The name is matched ignoring case, and part of the name is enough.
    System.out.println("\"add to cart\": " + page.getByRole(AriaRole.BUTTON, new
Page.GetByRoleOptions().setName("add to cart")).count());
    // setExact(true) wants the whole name with the same case.
    System.out.println("\"add to cart\" exact: " + page.getByRole(AriaRole.BUTTON,
        new Page.GetByRoleOptions().setName("add to cart").setExact(true)).count());
    System.out.println("disabled buttons: " + page.getByRole(AriaRole.BUTTON, new
Page.GetByRoleOptions().setDisabled(true)).count());

    page.getByRole(AriaRole.SEARCHBOX, new Page.GetByRoleOptions().setName("Search")).fill("lamp");
    page.getByRole(AriaRole.COMBOBOX, new Page.GetByRoleOptions().setName("Sort by")).selectOption("high");
    assertThat(page.getByRole(AriaRole.LISTITEM)).hasCount(1);
    System.out.println("left after search: " +
page.getByRole(AriaRole.LISTITEM).getByRole(AriaRole.HEADING).textContent());
  }
}
```

Run it: `mvn test`

Lesson: getByRole: the first choice

Labels, placeholders, text, titles and test ids

src/test/java/com/brainy/GetByTest.java

```
package com.brainy;

import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;
import java.util.regex.Pattern;
```

```java
import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class GetByTest {
  @Test
  void labelsPlaceholdersTextTitlesAndTestIds(Page page) {
    page.navigate("/login");
    page.getByLabel("Email").fill("asha@shop.example");
    page.getByLabel("Password").fill("s3cret");
    page.getByText("Remember me").click();          // clicking a label ticks its box
    assertThat(page.getByLabel("Remember me")).isChecked();
    page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Sign in")).click();

    assertThat(page.getByText("Hello, Asha")).isVisible();
    assertThat(page.getByTestId("cart-count")).hasText("0");
    assertThat(page.getByPlaceholder("Search products")).isEditable();
    assertThat(page.getByTestId(Pattern.compile("^product-"))).hasCount(5);   // all product cards

    page.navigate("/frames");
    System.out.println("iframes by title: " + page.getByTitle("Payment form").count());
  }
}
```

Run it: `mvn test`

Lesson: Labels, placeholders, text, alt text, titles and test ids

CSS and XPath

src/test/java/com/brainy/CssTest.java

```java
package com.brainy;

import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class CssTest {
  @Test
  void cssAndXpathWhenNothingBetterFits(Page page) {
    page.navigate("/products");
    assertThat(page.locator("li.card")).hasCount(5);

    System.out.println("CSS prices: " + page.locator("li.card .price").allTextContents());
    System.out.println("CSS attribute: " + page.locator("[data-testid=\"product-3\"] h3").textContent());
    System.out.println("CSS :has-text(): " + page.locator("li.card:has-text(\"mug\") h3").textContent());
    System.out.println("XPath: " + page.locator("//li[h3=\"Desk
lamp\"]//span[@class=\"price\"]").textContent());
    System.out.println("id: " + page.locator("#sort").inputValue());
  }
}
```

Run it: `mvn test`

Lesson: CSS and XPath

filter, first, nth, last, and, or

src/test/java/com/brainy/FilterTest.java

```java
package com.brainy;

import com.microsoft.playwright.Locator;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class FilterTest {
  @Test
```

```
  void narrowDownWithFilterChainingAndAndOr(Page page) {
    page.navigate("/products");
    Locator cards = page.getByRole(AriaRole.LISTITEM);
    assertThat(cards).hasCount(5);

    Locator mug = cards.filter(new Locator.FilterOptions().setHasText("Coffee mug"));
    System.out.println("mug price: " + mug.locator(".price").textContent());

    Locator kitchen = cards.filter(new Locator.FilterOptions().setHasText("Kitchen"));
    System.out.println("kitchen: " + kitchen.getByRole(AriaRole.HEADING).allTextContents());

    Locator soldOut = cards.filter(new Locator.FilterOptions()
        .setHas(page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Out of stock"))));
    System.out.println("sold out: " + soldOut.getByRole(AriaRole.HEADING).textContent());

    Locator inStock = cards.filter(new Locator.FilterOptions()
        .setHasNot(page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setDisabled(true))));
    System.out.println("in stock: " + inStock.count());

    System.out.println("first: " + cards.first().getByRole(AriaRole.HEADING).textContent());
    System.out.println("nth(2): " + cards.nth(2).getByRole(AriaRole.HEADING).textContent());
    System.out.println("last: " + cards.last().getByRole(AriaRole.HEADING).textContent());

    Locator anyButton = page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Add to cart"))
        .or(page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Out of stock")));
    System.out.println("or: " + anyButton.count());
    Locator disabledButton = page.getByRole(AriaRole.BUTTON).and(page.locator("[disabled]"));
    System.out.println("and: " + disabledButton.textContent());
  }
}
```

Run it: `mvn test`

Lesson: Filtering and chaining

Strict mode: one action, one element

src/test/java/com/brainy/StrictTest.java

```
package com.brainy;

import com.microsoft.playwright.Locator;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.PlaywrightException;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;
import static org.junit.jupiter.api.Assertions.assertThrows;

@UsePlaywright(ShopOptions.class)
public class StrictTest {
  @Test
  void strictModeRefusesToGuess(Page page) {
    page.navigate("/products");
    Locator add = page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Add to cart"));
    assertThat(add).hasCount(4);              // counting many is fine

    PlaywrightException error = assertThrows(PlaywrightException.class, () -> add.click());   // acting on
many is not
    System.out.println(String.join("\n", java.util.Arrays.asList(error.getMessage().split("\n")).subList(0,
5)));
  }
}
```

Run it: `mvn test`

Lesson: Strict mode

Working on one row of the orders table

`page.navigate("http://localhost:4173/orders");`	Open the orders page. It shows 5 of 7 orders per page.
`page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Total")).click();`	Click the Total column header to sort by total, smallest first.

`page.getByRole(AriaRole.ROW).filter(new Locator.FilterOptions().setHasText("John"))` `.getByRole(AriaRole.BUTTON, new Locator.GetByRoleOptions().setName("Delete")).click();`	Find the row that mentions John, then the Delete button inside that row only.
`assertThat(page.getByRole(AriaRole.STATUS)).hasText("Order #1004 deleted");`	The status message confirms which order went away.
`page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Next")).click();`	Go to the next page of the table.
`assertThat(page.getByText("Page 2 of 2")).isVisible();`	Check the page counter by its text.

Lesson: Lists and tables

Read every row of a table, then act on one row

src/test/java/com/brainy/TableTest.java

```java
package com.brainy;

import com.microsoft.playwright.Locator;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class TableTest {
  @Test
  void readATableAndActOnOneRow(Page page) {
    page.navigate("/orders");
    Locator rows = page.getByRole(AriaRole.ROW)
        .filter(new Locator.FilterOptions().setHas(page.getByRole(AriaRole.CELL)));   // skip the header row
    assertThat(rows).hasCount(5);

    for (Locator row : rows.all()) {
      System.out.println(row.getByRole(AriaRole.CELL).allTextContents());
    }

    Locator meera = rows.filter(new Locator.FilterOptions().setHasText("Meera"));
    meera.getByRole(AriaRole.BUTTON, new Locator.GetByRoleOptions().setName("Delete")).click();
    assertThat(page.getByRole(AriaRole.STATUS)).hasText("Order #1003 deleted");
    assertThat(rows.filter(new Locator.FilterOptions().setHasText("Meera"))).hasCount(0);
  }
}
```

Run it: `mvn test`

Lesson: Lists and tables

TABLES

Table: html · role · example locator

HTML	Role	Example locator
<button>, <input type=submit>	button	getByRole('button', { name: 'Sign in' })
<a href>	link	getByRole('link', { name: 'Orders' })
<h1> to <h6>	heading	getByRole('heading', { level: 1 })
<input type=text>, <textarea>	textbox	getByRole('textbox', { name: 'Email' })
<input type=search>	searchbox	getByRole('searchbox')
<select>	combobox	getByRole('combobox', { name: 'Sort by' })
<input type=checkbox> and radio	checkbox, radio	getByRole('checkbox', { checked: true })
, <tr>, <td>	listitem, row, cell	getByRole('row').filter(...)

Lesson: getByRole: the first choice

Table: locator · finds · good for

Locator	Finds	Good for
getByLabel	a form box by its <label> or aria-label	every form
getByPlaceholder	a box by its hint text	boxes with no label
getByText	the smallest element that contains the text	messages, paragraphs, labels
getByAltText	an image by its alt text	logos, product pictures
getByTitle	an element by its title attribute	icons, iframes
getByTestId	an element by data-testid	text that changes or is translated

Lesson: Labels, placeholders, text, alt text, titles and test ids

Table: you want · better choice · fragile choice

You want	Better choice	Fragile choice
the Sign in button	getByRole('button', { name: 'Sign in' })	locator('#login > button.btn-primary')
the email box	getByLabel('Email')	locator('//form/div[1]/input')
the price of the mug	getByRole('listitem').filter({ hasText: 'Coffee mug' }).getByText(/₹/)	locator('li:nth-child(1) span:nth-child(3)')

Lesson: CSS and XPath

Table: method · keeps

Method	Keeps
filter(new FilterOptions().setHasText("x"))	elements that contain the text somewhere inside
setHasNotText("x")	elements that do not contain the text
setHas(locator)	elements that contain an element the locator finds
setHasNot(locator)	elements that do not contain it
setVisible(true)	only visible elements
first(), last(), nth(i)	one element by position, nth counts from 0
a.and(b), a.or(b)	elements both match, or either matches

Lesson: Filtering and chaining

TIPS AND TRAPS

IN REAL TEAMS If getByRole cannot find a button that you can see, the page is probably missing a role or a name, and a screen reader user cannot use it either. That is a real accessibility bug worth reporting.

Lesson: getByRole: the first choice

IN REAL TEAMS Many companies use another attribute for test ids, such as data-qa or data-test. Tell Playwright once: `playwright.selectors().setTestIdAttribute("data-qa")`. After that, getByTestId reads the new attribute.

Lesson: Labels, placeholders, text, alt text, titles and test ids

COMMON MISTAKE Long XPaths copied from the browser tools, such as /html/body/div[2]/div/form/input, break as soon as one box is added to the page. Recorded tests from older tools are full of them, and they are the first thing to replace when a team moves to Playwright.

Lesson: CSS and XPath

COMMON MISTAKE first(), last() and nth() depend on the order of the page. When the order changes, they quietly pick another element. Prefer a filter by text when you can.

Lesson: Filtering and chaining

3.2 Actions and auto-waiting

An interactive auto-waiting lab, a real click on a covered, disabled and moving button, the actionability table from the docs, the call log of a timeout, force and trial, click options and hover, fill versus pressSequentially, a recorded walk through a feedback form, every form control, drag and drop by dragTo and by hand, and waits that replace sleeps.

KEY IDEAS

- **Auto-waiting** Checks before every action. No sleeps needed.
- **Timeouts** The error says which check failed. Read it before using force.
- **Typing** fill for most boxes. pressSequentially for key by key.
- **Forms** check, selectOption, dragTo. Found by their labels.
- **Remember** Never sleep. Every action waits until the element is ready, and when you need more, wait for the exact thing you need.

CODE

Click a button that is covered, disabled and moving

src/test/java/com/brainy/AutoWaitTest.java

```java
package com.brainy;

import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class AutoWaitTest {
  @Test
  void actionsWaitUntilTheElementIsReady(Page page) {
    page.navigate("/widgets");
    // A loading cover hides the page for 0.8 s, and Pay now is disabled for 1 s.
    page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Pay now")).click();
    assertThat(page.getByText("Payment done")).isVisible();

    // The offer appears 1.2 s after the click. The check waits for it.
    page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Load offers")).click();
    assertThat(page.getByTestId("offer")).hasText("Buy 2 mugs, get 1 free");

    // This button slides in when the page opens. click waits until it stops moving.
    page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Catch me")).click();
    assertThat(page.getByText("Caught it")).isVisible();
  }
}
```

Run it: `mvn test`

Lesson: Auto-waiting: no more sleeps

A click that can never happen, and its call log

src/test/java/com/brainy/CallLogTest.java

```java
package com.brainy;

import com.microsoft.playwright.Locator;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.TimeoutError;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;
import java.util.stream.Collectors;

import static org.junit.jupiter.api.Assertions.assertThrows;

@UsePlaywright(ShopOptions.class)
public class CallLogTest {
  @Test
```

```
  void whenItNeverGetsReadyTheErrorSaysWhy(Page page) {
    page.navigate("/products");
    Locator soldOut = page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Out of stock"));

    TimeoutError error = assertThrows(TimeoutError.class, () -> soldOut.click(new
Locator.ClickOptions().setTimeout(2000)));
    System.out.println(error.getClass().getSimpleName());
    String message = error.getMessage();
    // Java wraps the driver error and its stack, so print from the call log on.
    System.out.println(message.substring(message.indexOf("Call
log:")).lines().limit(6).collect(Collectors.joining("\n")));
  }
}
```

Run it: `mvn test`

Lesson: When waiting is not enough

Hover and focus

src/test/java/com/brainy/HoverTest.java

```
package com.brainy;

import com.microsoft.playwright.Locator;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class HoverTest {
  @Test
  void hoverAndFocusShowTheTooltip(Page page) {
    page.navigate("/forms");
    Locator help = page.getByText("❓ Help");
    Locator tooltip = page.getByRole(AriaRole.TOOLTIP);
    assertThat(tooltip).isHidden();

    help.hover();                              // the mouse rests on it
    assertThat(tooltip).hasText("We reply within two working days");

    page.mouse().move(0, 0);                   // move the mouse away
    assertThat(tooltip).isHidden();

    help.focus();                              // keyboard users reach it with Tab
    assertThat(tooltip).isVisible();
    System.out.println("tooltip: " + tooltip.textContent());
  }
}
```

Run it: `mvn test`

Lesson: Clicking, hovering and focusing

fill, pressSequentially and keyboard shortcuts

src/test/java/com/brainy/KeysTest.java

```
package com.brainy;

import com.microsoft.playwright.Locator;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class KeysTest {
  @Test
  void fillTypeKeyByKeyAndPressShortcuts(Page page) {
    page.navigate("/products");
    Locator search = page.getByPlaceholder("Search products");
```

```
    assertThat(page.getByRole(AriaRole.LISTITEM)).hasCount(5);

    search.fill("desk");                                    // sets the whole value at once
    System.out.println("after fill: " + search.inputValue() + " | " +
page.getByRole(AriaRole.LISTITEM).count() + " card");

    search.clear();
    search.pressSequentially("mug", new Locator.PressSequentiallyOptions().setDelay(50));   // one key at a
time
    System.out.println("after typing: " + search.inputValue());

    search.press("Control+A");
    search.press("Backspace");
    System.out.println("after select all and delete: " + search.inputValue().length() + " characters");

    page.navigate("/forms");
    page.keyboard().press("Control+K");                     // a shortcut for the whole page
    assertThat(page.getByText("Search opened")).isVisible();
  }
}
```

Run it: `mvn test`

Lesson: Typing and keys

Filling the feedback form

`page.navigate("http://localhost:4173/forms");`	Open the feedback form.
`page.getByLabel("Your name").fill("Asha");`	Type the name into the box labelled Your name.
`page.getByLabel("Topic").selectOption("delivery");`	Choose Delivery in the Topic drop down.
`page.getByLabel("Happy").check();`	Pick the Happy radio button.
`page.getByLabel("Message").fill("The parcel came a day early.");`	Write a short message.
`page.getByLabel("Send me the newsletter").check();`	Tick the newsletter box.
`page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Send feedback")).click();`	Send the form. Playwright scrolls the button into view by itself.
`assertThat(page.getByRole(AriaRole.STATUS)).containsText("Topic: delivery, mood: happy");`	The thank you line shows every value the page received.

Lesson: Checkboxes, radio buttons, drop downs and dates

Text boxes, drop downs, radio buttons, checkboxes, dates and sliders

src/test/java/com/brainy/FormTest.java

```
package com.brainy;

import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import com.microsoft.playwright.options.SelectOption;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class FormTest {
  @Test
  void fillEveryKindOfFormControl(Page page) {
    page.navigate("/forms");
    page.getByLabel("Your name").fill("Asha");
    page.getByLabel("Topic").selectOption("delivery");                          // by value
    page.getByLabel("Topic").selectOption(new SelectOption().setLabel("Payment"));  // or by the text you
see
    page.getByLabel("Happy").check();                                           // a radio button
    page.getByLabel("Order date").fill("2026-10-01");                           // dates use yyyy-mm-dd
    page.getByLabel("Message").fill("The parcel came a day early.");
    page.getByLabel("Send me the newsletter").setChecked(true);
    page.getByLabel("Volume").fill("8");                                        // a slider
```

```
    System.out.println("topic: " + page.getByLabel("Topic").inputValue());
    System.out.println("happy checked: " + page.getByLabel("Happy").isChecked());
    System.out.println("date: " + page.getByLabel("Order date").inputValue());
    System.out.println("volume: " + page.getByLabel("Volume").inputValue());

    page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Send feedback")).click();
    assertThat(page.getByRole(AriaRole.STATUS)).hasText(
        "Thanks Asha! Topic: payment, mood: happy, files: no files, newsletter: yes");
  }
}
```

Run it: `mvn test`

Lesson: Checkboxes, radio buttons, drop downs and dates

Drag and drop with dragTo, and by hand with the mouse

src/test/java/com/brainy/DragTest.java

```
package com.brainy;

import com.microsoft.playwright.Locator;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class DragTest {
  @Test
  void dragTasksToTheDoneLane(Page page) {
    page.navigate("/forms");
    Locator todo = page.getByRole(AriaRole.LIST, new Page.GetByRoleOptions().setName("To do"));
    Locator done = page.getByRole(AriaRole.LIST, new Page.GetByRoleOptions().setName("Done"));

    todo.getByText("Pack the order").dragTo(done);
    assertThat(done.getByRole(AriaRole.LISTITEM)).hasText(new String[] {"Pack the order"});

    // The same by hand with the mouse, for widgets that need finer control.
    todo.getByText("Print the label").hover();
    page.mouse().down();
    done.hover();
    done.hover();            // a second move, so the page gets its dragover event
    page.mouse().up();

    assertThat(done.getByRole(AriaRole.LISTITEM)).hasCount(2);
    System.out.println("Done lane: " + done.getByRole(AriaRole.LISTITEM).allTextContents());
    System.out.println("To do lane: " + todo.getByRole(AriaRole.LISTITEM).count() + " tasks");
  }
}
```

Run it: `mvn test`

Lesson: Drag and drop and the mouse

Wait for a response, an element or a condition

src/test/java/com/brainy/WaitTest.java

```
package com.brainy;

import com.microsoft.playwright.Locator;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.Response;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import com.microsoft.playwright.options.WaitForSelectorState;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class WaitTest {
  @Test
  void waitForWhatYouNeedNeverForAFixedTime(Page page) {
```

```
    // Start waiting for the API answer before the page asks for it.
    Response answer = page.waitForResponse(r -> r.url().contains("/api/products"), () ->
page.navigate("/products"));
    System.out.println("API answered with status " + answer.status());

    page.getByText("Loading products…").waitFor(new
Locator.WaitForOptions().setState(WaitForSelectorState.DETACHED));
    System.out.println("cards now: " + page.getByRole(AriaRole.LISTITEM).count());

    page.navigate("/widgets");
    page.waitForFunction("() => !document.querySelector('#cover')");
    System.out.println("loading cover left: " + page.locator("#cover").count());
    assertThat(page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Pay now"))).isEnabled();
  }
}
```

Run it: `mvn test`

Lesson: Waiting without sleeping

TABLES

Table: action · visible · stable · receives events

Action	Visible	Stable	Receives events	Enabled	Editable
click, dblclick, check, uncheck, setChecked, tap	✓	✓	✓	✓	
hover, dragTo	✓	✓	✓		
screenshot	✓	✓			
fill, clear	✓			✓	✓
selectOption	✓			✓	
selectText	✓				
scrollIntoViewIfNeeded		✓			
focus, blur, press, pressSequentially, setInputFiles, dispatchEvent					

Lesson: Auto-waiting: no more sleeps

Table: option · what it does · when to use it

Option	What it does	When to use it
setForce(true)	skips the checks that are not essential, such as receives events	a custom widget that hides its real input under a styled label
setTrial(true)	runs every check but does not click	to wait until something could be clicked, without clicking it
setTimeout(5000)	a different timeout for this one action	one action that is known to be slow

Lesson: When waiting is not enough

Table: you want · code

You want	Code
a double click	dblclick()
a right click	click(new Locator.ClickOptions().setButton(MouseButton.RIGHT))
Shift and click	setModifiers(Arrays.asList(KeyboardModifier.SHIFT))
Control on Windows and Linux, Command on macOS	KeyboardModifier.CONTROL_OR_META
a click at the top left corner	setPosition(0, 0)
the mouse resting on it	hover()
a tap on a touch screen	tap() with setHasTouch(true) on the context

Lesson: Clicking, hovering and focusing

Table: method · sends · use for

Method	Sends	Use for
fill(text)	the whole value at once	almost every text box
pressSequentially(text)	one key event per character	auto complete, masks that format as you type
press(key)	one key or a combination	Enter, Tab, Escape, ArrowDown, Control+A
keyboard.press(key)	a key to whatever has focus	page shortcuts such as Control+K
clear()	an empty value	emptying a box

Lesson: Typing and keys

Table: control · method · read it back

Control	Method	Read it back
text box, text area	fill	inputValue
checkbox, radio button	check, uncheck, setChecked	isChecked or toBeChecked
drop down (select)	selectOption by value, label or index, or a list for multi select	inputValue or toHaveValue
date, time, color, range	fill with the standard format, such as 2026-10-01	inputValue
file input	setInputFiles	the files list

Lesson: Checkboxes, radio buttons, drop downs and dates

Table: wait for · method

Wait for	Method
an element to appear, hide or go away	locator.waitFor(new Locator.WaitForOptions().setState(...))
a new URL	page.waitForURL
a network response	page.waitForResponse(predicate, action)
anything the page can compute	page.waitForFunction
a popup, download or dialog	page.waitForPopup and friends

Lesson: Waiting without sleeping

TIPS AND TRAPS

COMMON MISTAKE Reaching for force is often a sign the test found the wrong element, or that a real user could not click it either. Read the call log first.

Lesson: When waiting is not enough

NEW IN PLAYWRIGHT Old tutorials use type(). It was renamed pressSequentially in Playwright 1.38 and the old name is deprecated. If you see it in a code base, it does the same thing.

Lesson: Typing and keys

IN REAL TEAMS Many teams add a lint rule or a code review check that rejects waitForTimeout. A suite with 200 tests and a 2 second sleep in each spends almost 7 minutes doing nothing on every run.

Lesson: Waiting without sleeping

3.3 Assertions that wait for the page

How a web-first assertion retries, a plain read against a retrying check, fifteen checks on one page, text and list checks for sorting, negative checks, a real failure report, soft assertions in all four languages, plain value checks on API data, expect.poll and toPass, and a recorded walk through a search.

KEY IDEAS

- **Web-first** Retries until true. 5 seconds by default.

- **Text** Exact or contains. Lists check order too.
- **Negative** Waits for things to go. toHaveCount(0) for none.
- **Soft** Keep going. Report every problem.
- **Remember** Check the page with assertions that retry, and never read a value first and check it later.

CODE

A web-first check against a plain read

src/test/java/com/brainy/RetryTest.java

```java
package com.brainy;

import com.microsoft.playwright.Locator;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class RetryTest {
  @Test
  void aWebFirstCheckWaitsAPlainReadDoesNot(Page page) {
    page.navigate("/widgets");
    page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Load offers")).click();
    Locator offer = page.getByTestId("offer");      // shows up 1.2 s after the click

    System.out.println("isVisible() right now: " + offer.isVisible());   // a snapshot, no waiting
    assertThat(offer).isVisible();                                       // retries until it shows
    System.out.println("isVisible() after the check: " + offer.isVisible());
  }
}
```

Run it: `mvn test`

Lesson: Web-first assertions retry

Fifteen checks on the sign in page

src/test/java/com/brainy/ManyChecksTest.java

```java
package com.brainy;

import com.microsoft.playwright.Locator;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;
import java.util.regex.Pattern;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class ManyChecksTest {
  @Test
  void oneCheckForEachThingAUserCanNotice(Page page) {
    page.navigate("/login");
    assertThat(page).hasTitle("Sign in · Brainy Shop");
    assertThat(page).hasURL(Pattern.compile("/login$"));

    Locator email = page.getByLabel("Email");
    assertThat(email).isEmpty();
    assertThat(email).isEditable();
    assertThat(email).hasAttribute("type", "email");
    assertThat(email).hasRole(AriaRole.TEXTBOX);
    email.fill("asha@shop.example");
    assertThat(email).hasValue("asha@shop.example");
    assertThat(email).isFocused();

    assertThat(page.getByRole(AriaRole.CHECKBOX, new Page.GetByRoleOptions().setName("Remember
me"))).not().isChecked();
    assertThat(page.getByRole(AriaRole.ALERT)).isHidden();
```

```
    Locator signIn = page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Sign in"));
    assertThat(signIn).isEnabled();
    assertThat(signIn).hasCSS("background-color", "rgb(22, 163, 74)");
    assertThat(signIn).isInViewport();
    assertThat(page.getByRole(AriaRole.FORM)).hasAccessibleName("Sign in");
    System.out.println("all 15 checks passed");
  }
}
```

Run it: `mvn test`

Lesson: The assertions you will use every day

toHaveText and toContainText for one element and for lists

src/test/java/com/brainy/TextTest.java

```
package com.brainy;

import com.microsoft.playwright.Locator;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.assertions.LocatorAssertions;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;
import java.util.regex.Pattern;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class TextTest {
  @Test
  void textChecksForOneElementAndForAList(Page page) {
    page.navigate("/products");
    Locator names = page.getByRole(AriaRole.LISTITEM).getByRole(AriaRole.HEADING);

    // A list: every element, in this order.
    assertThat(names).hasText(new String[] {"Coffee mug", "Desk lamp", "Gel pen", "Notebook", "Water
bottle"});
    // A list: some elements contain these texts, in this order.
    assertThat(names).containsText(new String[] {"mug", "pen"});

    assertThat(page.getByRole(AriaRole.HEADING, new Page.GetByRoleOptions().setLevel(1)))
        .hasText("products", new LocatorAssertions.HasTextOptions().setIgnoreCase(true));
    assertThat(page.getByTestId("product-3")).containsText("₹1,299");
    assertThat(page.getByTestId("product-3")).hasText(Pattern.compile("Desk lamp.*Home"));

    page.getByRole(AriaRole.COMBOBOX, new Page.GetByRoleOptions().setName("Sort by")).selectOption("low");
    assertThat(names).hasText(new String[] {"Gel pen", "Notebook", "Coffee mug", "Water bottle", "Desk
lamp"});
    System.out.println("sorted: " + names.allTextContents());
  }
}
```

Run it: `mvn test`

Lesson: Checking text and lists

Checking a search, one assertion at a time

`page.navigate("http://localhost:4173/products");`	Open the products page.
`assertThat(page.getByRole(AriaRole.LISTITEM)).hasCount(5);`	The check waits until the slow product list has loaded.
`page.getByPlaceholder("Search products").fill("bottle");`	Search for bottle.
`assertThat(page.getByRole(AriaRole.LISTITEM)).containsText("Water bottle");`	One card is left, and it is the right one.
`assertThat(page.getByPlaceholder("Search products")).hasValue("bottle");`	The search box keeps what was typed.
`page.getByPlaceholder("Search products").fill("phone");`	Search for something the shop does not sell.
`assertThat(page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Add to cart"))).hasCount(0);`	No Add to cart buttons are left. A negative check also waits.

Lesson: Checking that something is not there

Checks that something is gone

src/test/java/com/brainy/NegativeTest.java

```
package com.brainy;

import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class NegativeTest {
  @Test
  void negativeChecksWaitToo(Page page) {
    page.navigate("/products");
    assertThat(page.getByText("Loading products…")).not().isVisible();   // waits until it goes away
    assertThat(page.getByText("No products match your search")).hasCount(0);

    page.getByPlaceholder("Search products").fill("phone");
    assertThat(page.getByRole(AriaRole.LISTITEM)).hasText("No products match your search");
    assertThat(page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Add to
cart"))).hasCount(0);
    assertThat(page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Add to
cart"))).not().isAttached();
    System.out.println("nothing to buy when searching for phone");
  }
}
```

Run it: `mvn test`

Lesson: Checking that something is not there

What a failing check looks like

src/test/java/com/brainy/FailingTest.java

```
package com.brainy;

import com.microsoft.playwright.Page;
import com.microsoft.playwright.assertions.LocatorAssertions;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class FailingTest {
  @Test
  void aFailingCheckExplainsItself(Page page) {
    page.navigate("/products");
    assertThat(page.getByRole(AriaRole.LISTITEM)).hasCount(5);
    // Java has no message argument. The test name and a comment carry the intent.
    assertThat(page.getByTestId("product-2"))
        .containsText("₹149", new LocatorAssertions.ContainsTextOptions().setTimeout(2000));
  }
}
```

Run it: `mvn test`

Lesson: Reading a failure

Soft checks: keep going, then fail with every problem

src/test/java/com/brainy/SoftTest.java

```
package com.brainy;

import com.microsoft.playwright.Page;
import com.microsoft.playwright.assertions.LocatorAssertions;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;
```

```java
import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;
import static org.junit.jupiter.api.Assertions.assertAll;

@UsePlaywright(ShopOptions.class)
public class SoftTest {
  @Test
  void softChecksReportEveryProblemAtOnce(Page page) {
    page.navigate("/products");
    assertThat(page.getByRole(AriaRole.LISTITEM)).hasCount(5);
    LocatorAssertions.ContainsTextOptions quick = new
LocatorAssertions.ContainsTextOptions().setTimeout(1000);

    // JUnit 5 runs every check in the group and reports all failures together.
    assertAll("product cards",
        () -> assertThat(page.getByTestId("product-1")).containsText("₹25", quick),          // wrong on
purpose
        () -> assertThat(page.getByTestId("product-4")).containsText("Add to cart", quick),  // wrong on
purpose
        () -> assertThat(page.getByTestId("product-5")).containsText("₹349"));               // right
  }
}
```

Run it: `mvn test`

Lesson: Soft assertions

Plain checks on data you already have

src/test/java/com/brainy/ValueTest.java

```java
package com.brainy;

import com.google.gson.Gson;
import com.google.gson.JsonArray;
import com.google.gson.JsonElement;
import com.microsoft.playwright.APIResponse;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import org.junit.jupiter.api.Test;
import java.util.ArrayList;
import java.util.List;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;
import static org.junit.jupiter.api.Assertions.*;

@UsePlaywright(ShopOptions.class)
public class ValueTest {
  @Test
  void plainValueChecksForDataYouAlreadyHave(Page page) {
    APIResponse response = page.request().get("/api/products?delay=0");
    assertThat(response).isOK();                    // a Playwright check for API answers
    JsonArray products = new Gson().fromJson(response.text(), JsonArray.class);   // Gson comes with
Playwright

    assertEquals(5, products.size());               // plain JUnit checks
    List<String> names = new ArrayList<>();
    for (JsonElement p : products) names.add(p.getAsJsonObject().get("name").getAsString());
    assertTrue(names.contains("Desk lamp"));
    assertEquals("Gel pen", products.get(0).getAsJsonObject().get("name").getAsString());
    assertTrue(products.get(2).getAsJsonObject().get("price").getAsInt() > 1000);
    int cheapest = Integer.MAX_VALUE;
    for (JsonElement p : products) cheapest = Math.min(cheapest,
p.getAsJsonObject().get("price").getAsInt());
    System.out.println("cheapest: " + cheapest);
  }
}
```

Run it: `mvn test`

Lesson: Plain values, polling and blocks

TABLES

Table: checks · java

Checks	Java
visible or hidden	isVisible, isHidden
text	hasText, containsText
how many	hasCount
box value	hasValue, hasValues, isEmpty
state	isChecked, isEnabled, ...
attributes and style	hasAttribute, hasClass, ...
accessibility	hasRole, ...
the page	hasTitle, hasURL
an API answer	isOK
pictures and structure	matchesAriaSnapshot

Lesson: The assertions you will use every day

Table: assertion · with a string · with a list of strings

Assertion	With a string	With a list of strings
toHaveText	the whole text must match	exactly these elements, in this order
toContainText	the text must contain it	some elements, in this order, containing these parts

Lesson: Checking text and lists

TIPS AND TRAPS

COMMON MISTAKE A rule that catches many flaky tests: never put await inside expect. Write `assertThat(locator).isVisible()`, not `assertTrue(locator.isVisible())`.

Lesson: Web-first assertions retry

REMEMBER Change the timeout of one check with its timeout option, or of all checks: `PlaywrightAssertions.setDefaultAssertionTimeout(10_000)`. Raise it for a slow test environment, not to hide a slow page.

Lesson: Reading a failure

IN REAL TEAMS Soft checks are handy for a page with many independent fields, such as an invoice or a profile. Do not use them for steps the rest of the test depends on, such as signing in.

Lesson: Soft assertions

STEP 4 The test runner

4.1 The test runner: organise, tag, parameterise and configure

Which runner each language uses, what happens in a test run, groups, tags, skip and expected failures in all four languages, running only smoke tests, parameterised search tests, and one test run as a desktop and as a phone from configuration.

KEY IDEAS

- **Runner** Finds and runs tests. One per language.
- **Tags** Run a slice. Smoke now, regression tonight.
- **Data** One test, many rows. Each row is its own result.
- **Config** Settings in one place. Projects for each device.

- **Remember** Tests say what to check. The runner and its configuration decide where, how and with what settings they run.

CODE

Group, tag, skip and mark known failures

src/test/java/com/brainy/ProductsPageTest.java

```java
package com.brainy;

import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Disabled;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Tag;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
@DisplayName("Products page")
public class ProductsPageTest {
  @Test
  @Tag("smoke")
  @DisplayName("shows five products")
  void showsFiveProducts(Page page) {
    page.navigate("/products");
    assertThat(page.getByRole(AriaRole.LISTITEM)).hasCount(5);
  }

  @Test
  @Tag("regression")
  @DisplayName("sorts by price")
  void sortsByPrice(Page page) {
    page.navigate("/products");
    page.getByLabel("Sort by").selectOption("low");
    assertThat(page.getByRole(AriaRole.LISTITEM).first()).containsText("Gel pen");
  }

  @Test
  @Disabled("the wish list is not built yet")
  void addsAProductToTheWishList(Page page) {
  }

  // JUnit has no "expected to fail". Teams disable the test and name the ticket: SHOP-42.
}
```

Run it: `mvn test`

Lesson: Grouping, tags and annotations

Run only the tests with one tag

src/test/java/com/brainy/ProductsPageTest.java

```java
package com.brainy;

import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Disabled;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Tag;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
@DisplayName("Products page")
public class ProductsPageTest {
  @Test
  @Tag("smoke")
  @DisplayName("shows five products")
```

```java
  void showsFiveProducts(Page page) {
    page.navigate("/products");
    assertThat(page.getByRole(AriaRole.LISTITEM)).hasCount(5);
  }

  @Test
  @Tag("regression")
  @DisplayName("sorts by price")
  void sortsByPrice(Page page) {
    page.navigate("/products");
    page.getByLabel("Sort by").selectOption("low");
    assertThat(page.getByRole(AriaRole.LISTITEM).first()).containsText("Gel pen");
  }

  @Test
  @Disabled("the wish list is not built yet")
  void addsAProductToTheWishList(Page page) {
  }

  // JUnit has no "expected to fail". Teams disable the test and name the ticket: SHOP-42.
}
```

Run it: `mvn test -Dgroups=smoke`

Lesson: Running a slice of the suite

One test, many rows of data

src/test/java/com/brainy/SearchTest.java

```java
package com.brainy;

import com.microsoft.playwright.Locator;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class SearchTest {
  @ParameterizedTest(name = "search for \"{0}\" finds {1}")
  @CsvSource({
      "mug, 1, Coffee mug",
      "o, 3, Coffee mug",
      "pen, 1, Gel pen",
      "BOTTLE, 1, Water bottle",
  })
  void search(String term, int count, String first, Page page) {   // the data first, then Page
    page.navigate("/products");
    page.getByPlaceholder("Search products").fill(term);
    Locator cards = page.getByRole(AriaRole.LISTITEM);
    assertThat(cards).hasCount(count);
    assertThat(cards.first()).containsText(first);
  }
}
```

Run it: `mvn test`

Lesson: Parameterised tests

Configuration that every test receives

src/test/java/com/brainy/ShopOptions.java

```java
package com.brainy;

import com.microsoft.playwright.Browser;
import com.microsoft.playwright.junit.Options;
import com.microsoft.playwright.junit.OptionsFactory;

// Shared settings for every test class that says @UsePlaywright(ShopOptions.class)
public class ShopOptions implements OptionsFactory {
  @Override
  public Options getOptions() {
```

```
    return new Options()
        .setHeadless(true)
        .setBrowserName("chromium")
        .setBaseUrl("http://localhost:4173")
        .setContextOptions(new Browser.NewContextOptions()
            .setLocale("en-IN")
            .setTimezoneId("Asia/Kolkata"));
  }
}
```

src/test/java/com/brainy/PhoneOptions.java

```
package com.brainy;

import com.microsoft.playwright.Browser;
import com.microsoft.playwright.junit.Options;
import com.microsoft.playwright.junit.OptionsFactory;

// The same shop, as a phone. setDeviceName uses Playwright's device list.
public class PhoneOptions implements OptionsFactory {
  @Override
  public Options getOptions() {
    return new Options()
        .setHeadless(true)
        .setBaseUrl("http://localhost:4173")
        .setDeviceName("Pixel 7")
        .setContextOptions(new Browser.NewContextOptions()
            .setLocale("en-IN")
            .setTimezoneId("Asia/Kolkata"));
  }
}
```

src/test/java/com/brainy/AboutYou.java

```
package com.brainy;

import com.microsoft.playwright.Page;

// A helper both test classes share.
public class AboutYou {
  static void show(String name, Page page) {
    page.navigate("/about-you");
    System.out.println(String.format("%-8s", name)
        + "| viewport " + page.viewportSize().width
        + " | language " + page.locator("#lang").textContent()
        + " | time zone " + page.locator("#tz").textContent());
  }
}
```

src/test/java/com/brainy/DesktopTest.java

```
package com.brainy;

import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import org.junit.jupiter.api.Test;

@UsePlaywright(ShopOptions.class)
public class DesktopTest {
  @Test
  void theConfigReachesEveryTest(Page page) { AboutYou.show("desktop", page); }
}
```

src/test/java/com/brainy/PhoneTest.java

```
package com.brainy;

import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import org.junit.jupiter.api.Test;

@UsePlaywright(PhoneOptions.class)
public class PhoneTest {
  @Test
  void theConfigReachesEveryTest(Page page) { AboutYou.show("phone", page); }
}
```

Run it: `mvn test`

Lesson: Configuration in one place

TABLES

Table: language · test runner · how playwright plugs in

Language	Test runner	How Playwright plugs in
Java	JUnit 5 or TestNG	JUnit 5: @UsePlaywright injects Page and friends (since 1.42). TestNG: you create them in hooks

Lesson: One library, a runner per language

Table: you want · java (junit 5)

You want	Java (JUnit 5)
a group	a test class with @DisplayName
a tag	@Tag("smoke")
skip	@Disabled("why")
skip on a condition	@EnabledOnOs, @DisabledIf...
known to fail	not built in, @Disabled with the ticket
broken, fix later	@Disabled
give it more time	@Timeout
run only this one	-Dtest=Class#method

Lesson: Grouping, tags and annotations

Table: setting · java junit

Setting	Java JUnit
base URL	setBaseUrl
browser	setBrowserName("firefox")
headed	setHeadless(false)
device	setDeviceName("Pixel 7")
locale and other context options	setContextOptions(...)
test timeout	@Timeout or junit.jupiter.execution.timeout
screenshots, video, trace	setTrace(...) and your own hooks

Lesson: Configuration in one place

TIPS AND TRAPS

COMMON MISTAKE A test.only left in the code silently skips every other test. Set forbidOnly in the CI config so such a run fails instead.

Lesson: Grouping, tags and annotations

IN REAL TEAMS In real projects the rows often come from a CSV or JSON file, for example 50 postcodes for a delivery check. Read the file at the top of the test file and loop over it, or use @CsvFileSource in JUnit and a list built from the file in pytest.

Lesson: Parameterised tests

4.2 Fixtures, hooks and set up once

The order hooks run in, the built in fixtures of each language, the life of a fixture, a signed in page fixture with a clean up, an automatic fixture that fails tests on JavaScript errors, and set up that checks the shop and creates data once per run.

KEY IDEAS

- **Hooks** Around every test. Simple and visible.
- **Fixtures** Only when asked for. Set up, use, tear down.
- **Automatic** Every test gets it. Guards and recordings.
- **Once per run** Health check, data. Before any test.
- **Remember** Put set up where it belongs: in the test if one test needs it, in a fixture if several do, and in a setup step if the whole run does.

CODE

Before and after hooks, and the order they run in

src/test/java/com/brainy/HooksTest.java

```
package com.brainy;

import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.*;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
@TestMethodOrder(MethodOrderer.OrderAnnotation.class)
public class HooksTest {
  @BeforeAll
  static void onceForThisClass() {
    System.out.println("@BeforeAll: once for this class");
  }

  @BeforeEach
  void openProducts(Page page, TestInfo info) {
    System.out.println("@BeforeEach: open the products page for " + info.getDisplayName());
    page.navigate("/products");
  }

  @AfterEach
  void done(TestInfo info) {
    System.out.println("@AfterEach: " + info.getDisplayName() + " done");
  }

  @AfterAll
  static void atTheEnd() {
    System.out.println("@AfterAll: once at the end");
  }

  @Test
  @Order(1)
  void showsFiveProducts(Page page) {
    assertThat(page.getByRole(AriaRole.LISTITEM)).hasCount(5);
  }

  @Test
  @Order(2)
  void findsTheLamp(Page page) {
    page.getByPlaceholder("Search products").fill("lamp");
    assertThat(page.getByRole(AriaRole.LISTITEM)).hasCount(1);
  }
}
```

Run it: `mvn test`

Lesson: Before and after hooks

A signed in page for every test that asks for it

src/test/java/com/brainy/SignedInTest.java

```
package com.brainy;

import com.microsoft.playwright.Page;
```

```java
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

// Every test class that extends this one starts signed in as Asha.
@UsePlaywright(ShopOptions.class)
public abstract class SignedInTest {
  @BeforeEach
  void signIn(Page page) {
    page.navigate("/login");
    page.getByLabel("Email").fill("asha@shop.example");
    page.getByLabel("Password").fill("s3cret");
    page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Sign in")).click();
    assertThat(page.getByText("Hello, Asha")).isVisible();
    System.out.println("set up: signed in");
  }

  @AfterEach
  void emptyCart(Page page) {
    page.request().delete("/api/cart");
    System.out.println("tear down: cart emptied");
  }
}
```

src/test/java/com/brainy/CartTest.java

```java
package com.brainy;

import com.microsoft.playwright.Locator;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

public class CartTest extends SignedInTest {
  @Test
  void addAMugToTheCart(Page page) {
    page.getByTestId("product-2").getByRole(AriaRole.BUTTON, new Locator.GetByRoleOptions().setName("Add to
cart")).click();
    assertThat(page.getByRole(AriaRole.STATUS)).hasText("Coffee mug added to your cart");
    assertThat(page.getByTestId("cart-count")).hasText("1");
    System.out.println("test: the mug is in the cart");
  }
}
```

Run it: `mvn test`

Lesson: Fixtures: set up on demand

An automatic fixture that fails a test when the page throws an error

src/test/java/com/brainy/NoPageErrorsTest.java

```java
package com.brainy;

import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import java.util.ArrayList;
import java.util.List;

import static org.junit.jupiter.api.Assertions.assertEquals;

// Every test class that extends this one fails when its page throws an error.
@UsePlaywright(ShopOptions.class)
public abstract class NoPageErrorsTest {
  private final List<String> errors = new ArrayList<>();

  @BeforeEach
  void listen(Page page) {
    page.onPageError(error -> errors.add(error.split("\n")[0]));
```

```
  }

  @AfterEach
  void noErrors(Page page) {
    page.waitForTimeout(100);   // Java hands over browser events only while a Playwright call runs
    assertEquals(List.of(), errors, "the page threw errors");
  }
}
```

src/test/java/com/brainy/PagesTest.java

```
package com.brainy;

import com.microsoft.playwright.Page;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

public class PagesTest extends NoPageErrorsTest {
  @Test
  void theProductsPageHasNoErrors(Page page) {
    page.navigate("/products");
    assertThat(page.getByRole(AriaRole.LISTITEM)).hasCount(5);
  }

  @Test
  void theWidgetsPageBreaks(Page page) {
    page.navigate("/widgets");
    page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Break the page")).click();
    assertThat(page.getByRole(AriaRole.HEADING, new Page.GetByRoleOptions().setName("Console and
errors"))).isVisible();
    // the test itself passes, the check after it finds the error
  }
}
```

Run it: `mvn test`

Lesson: Automatic fixtures

Set up once: check the shop and create test data

src/test/java/com/brainy/ApiOptions.java

```
package com.brainy;

import com.microsoft.playwright.APIRequest;
import com.microsoft.playwright.junit.Options;
import com.microsoft.playwright.junit.OptionsFactory;

// setBaseUrl is for pages. The request fixture reads its own options.
public class ApiOptions implements OptionsFactory {
  @Override
  public Options getOptions() {
    return new Options()
        .setHeadless(true)
        .setBaseUrl("http://localhost:4173")
        .setApiRequestOptions(new APIRequest.NewContextOptions().setBaseURL("http://localhost:4173"));
  }
}
```

src/test/java/com/brainy/PeopleTest.java

```
package com.brainy;

import com.google.gson.JsonParser;
import com.microsoft.playwright.APIRequest;
import com.microsoft.playwright.APIRequestContext;
import com.microsoft.playwright.APIResponse;
import com.microsoft.playwright.Playwright;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.RequestOptions;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.Test;
import java.util.Map;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;
```

```java
import static org.junit.jupiter.api.Assertions.assertEquals;

@UsePlaywright(ApiOptions.class)
public class PeopleTest {
  @BeforeAll                                          // once for this class
  static void shopData() {
    try (Playwright playwright = Playwright.create()) {
      APIRequestContext api = playwright.request().newContext(
          new APIRequest.NewContextOptions().setBaseURL("http://localhost:4173"));
      assertThat(api.get("/api/health")).isOK();
      APIResponse response = api.post("/api/people",
          RequestOptions.create().setData(Map.of("name", "Test Customer", "email", "test@shop.example")));
      assertEquals(201, response.status());
      System.out.println("setup: created person " +
JsonParser.parseString(response.text()).getAsJsonObject().get("id"));
    }
  }

  @Test
  void theTestDataIsThere(APIRequestContext request) {
    var people = JsonParser.parseString(request.get("/api/people").text()).getAsJsonArray();
    System.out.println("test: people in the shop: " + people);
    assertEquals(1, people.size());
  }
}
```

Run it: `mvn test`

Lesson: Set up once for the whole run

TABLES

Table: when · junit 5

When	JUnit 5
before each test	@BeforeEach
after each test	@AfterEach
once before all	@BeforeAll (static)
once after all	@AfterAll (static)

Lesson: Before and after hooks

Table: fixture · java @useplaywright

Fixture	Java @UsePlaywright
a fresh page	Page page
its browser context	BrowserContext browserContext
the shared browser	Browser browser
an API client	APIRequestContext request
which browser	browser.browserType().name()
Playwright itself	Playwright playwright

Lesson: Fixtures: set up on demand

Table: scope · junit 5

Scope	JUnit 5
each test	@BeforeEach
each worker process	not built in
the file or class	@BeforeAll
the whole run	LauncherSessionListener, or @BeforeAll in a base class

Lesson: Set up once for the whole run

TIPS AND TRAPS

IN REAL TEAMS Hooks run for every test in the file. Fixtures run only for tests that ask for them, and they can depend on other fixtures. As a suite grows, teams move set up from hooks into fixtures.

Lesson: Fixtures: set up on demand

4.3 Parallel runs, retries, timeouts and sharding

Four tests on two workers in each language, two tests kept apart with a lock, a flaky test that passes on retry, the five kinds of timeout with one that is too short, and a suite split into two shards.

KEY IDEAS

- **Workers** Tests at the same time. Each with its own browser.
- **Isolation** No shared data. Lock what must be shared.
- **Retries** Try again on failure. Flaky is still a bug.
- **Shards** Split across machines. Merge the reports.
- **Remember** Parallel runs only work when every test can run alone, in any order, next to any other test.

CODE

Four tests, two at a time

src/test/resources/junit-platform.properties

```
# Run test classes at the same time, on 2 threads. Tests inside one class stay in order.
junit.jupiter.execution.parallel.enabled = true
junit.jupiter.execution.parallel.mode.default = same_thread
junit.jupiter.execution.parallel.mode.classes.default = concurrent
junit.jupiter.execution.parallel.config.strategy = fixed
junit.jupiter.execution.parallel.config.fixed.parallelism = 2
```

src/test/java/com/brainy/ProductsTest.java

```
package com.brainy;

import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class ProductsTest {
  @Test
  void showsFiveProducts(Page page) {
    System.out.println("showsFiveProducts on " + Thread.currentThread().getName());
    page.navigate("/products");
    assertThat(page.getByRole(AriaRole.LISTITEM)).hasCount(5);
  }

  @Test
  void findsTheLamp(Page page) {
    System.out.println("findsTheLamp on " + Thread.currentThread().getName());
    page.navigate("/products");
    page.getByPlaceholder("Search products").fill("lamp");
    assertThat(page.getByRole(AriaRole.LISTITEM)).hasCount(1);
  }
}
```

src/test/java/com/brainy/OrdersTest.java

```
package com.brainy;

import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;
```

```java
import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class OrdersTest {
  @Test
  void pageOneShowsFiveOrders(Page page) {
    System.out.println("pageOneShowsFiveOrders on " + Thread.currentThread().getName());
    page.navigate("/orders");
    assertThat(page.getByRole(AriaRole.ROW)).hasCount(6);        // a header row and five orders
  }

  @Test
  void pageTwoIsTheLastPage(Page page) {
    System.out.println("pageTwoIsTheLastPage on " + Thread.currentThread().getName());
    page.navigate("/orders");
    page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Next")).click();
    assertThat(page.getByText("Page 2 of 2")).isVisible();
  }
}
```

Run it: `mvn test`

Lesson: Workers: tests at the same time

Two tests that must not run at the same time

src/test/java/com/brainy/AddMeeraTest.java

```java
package com.brainy;

import com.google.gson.JsonParser;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.RequestOptions;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.parallel.ResourceLock;
import java.util.Map;

import static org.junit.jupiter.api.Assertions.assertEquals;

@UsePlaywright(ShopOptions.class)
public class AddMeeraTest {
  @Test
  @ResourceLock("people")              // never at the same time as another test holding "people"
  void addMeera(Page page) {
    System.out.println("add Meera: start");
    int before = JsonParser.parseString(page.request().get("/api/people").text()).getAsJsonArray().size();
    page.request().post("/api/people", RequestOptions.create().setData(Map.of("name", "Meera", "email",
"meera@shop.example")));
    int after = JsonParser.parseString(page.request().get("/api/people").text()).getAsJsonArray().size();
    assertEquals(before + 1, after);
    System.out.println("add Meera: done");
  }
}
```

src/test/java/com/brainy/ProductsTest.java

```java
package com.brainy;

import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class ProductsTest {
  @Test                                // no lock, runs alongside
  void showsFiveProducts(Page page) {
    System.out.println("products: start");
    page.navigate("/products");
    assertThat(page.getByRole(AriaRole.LISTITEM)).hasCount(5);
    System.out.println("products: done");
```

```
  }
}
```

Run it: `mvn test`

Lesson: Parallel tests must not share data

A flaky test that passes on the second try

src/test/java/com/brainy/OfferTest.java

```
package com.brainy;

import com.microsoft.playwright.*;
import com.microsoft.playwright.assertions.LocatorAssertions;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.*;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

// Surefire reruns this class from the start, so it opens and closes Playwright itself.
public class OfferTest {
  private static int attempt = 0;      // the rerun happens in the same JVM
  static Playwright playwright;
  static Browser browser;
  BrowserContext context;
  Page page;

  @BeforeAll
  static void launch() {
    playwright = Playwright.create();
    browser = playwright.chromium().launch();
  }

  @BeforeEach
  void newPage() {
    context = browser.newContext(new Browser.NewContextOptions().setBaseURL("http://localhost:4173"));
    page = context.newPage();
  }

  @Test
  void theOfferShowsUp() {
    attempt++;
    System.out.println("attempt " + attempt);
    page.navigate("/widgets");
    page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Load offers")).click();
    // Flaky on purpose: the first try waits only 0.5 s, but the offer needs 1.2 s.
    double wait = attempt == 1 ? 500 : 5_000;
    assertThat(page.getByTestId("offer")).isVisible(new
LocatorAssertions.IsVisibleOptions().setTimeout(wait));
  }

  @AfterEach
  void closeContext() {
    context.close();
  }

  @AfterAll
  static void close() {
    playwright.close();
  }
}
```

Run it: `mvn test -Dsurefire.rerunFailingTestsCount=2`

Lesson: Retries and flaky tests

Timeouts that are long enough, and one that is too short

src/test/java/com/brainy/TimeoutsTest.java

```
package com.brainy;

import com.microsoft.playwright.Locator;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.assertions.LocatorAssertions;
import com.microsoft.playwright.assertions.PlaywrightAssertions;
```

```java
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.*;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
@TestMethodOrder(MethodOrderer.OrderAnnotation.class)
@Timeout(30)                                        // each test, in seconds
public class TimeoutsTest {
  @BeforeAll
  static void expectTimeout() {
    PlaywrightAssertions.setDefaultAssertionTimeout(5_000);   // each assertThat check
  }

  @BeforeEach
  void timeouts(Page page) {
    page.setDefaultTimeout(10_000);                 // each click, fill, check and so on
    page.setDefaultNavigationTimeout(15_000);       // each navigate
  }

  @Test
  @Order(1)
  void waitsLongEnoughForTheOffer(Page page) {
    page.navigate("/widgets");
    page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Load offers")).click();
    assertThat(page.getByTestId("offer")).hasText("Buy 2 mugs, get 1 free");   // the 5 s default is enough
  }

  @Test
  @Order(2)
  void givesUpTooEarly(Page page) {
    page.navigate("/widgets");
    page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Load offers")).click();
    assertThat(page.getByTestId("offer")).isVisible(new
LocatorAssertions.IsVisibleOptions().setTimeout(1_000));
  }

  @Test
  @Order(3)
  void aClickWithItsOwnTimeout(Page page) {
    page.navigate("/widgets");
    page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Pay now")).click(new
Locator.ClickOptions().setTimeout(3_000));
    assertThat(page.getByText("Payment done")).isVisible();
  }
}
```

Run it: `mvn test`

Lesson: Timeouts

Split the suite in two, as two CI machines would

src/test/java/com/brainy/ProductsTest.java

```java
package com.brainy;

import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class ProductsTest {
  @Test
  void showsFiveProducts(Page page) {
    System.out.println("showsFiveProducts on " + Thread.currentThread().getName());
    page.navigate("/products");
    assertThat(page.getByRole(AriaRole.LISTITEM)).hasCount(5);
  }

  @Test
```

```
  void findsTheLamp(Page page) {
    System.out.println("findsTheLamp on " + Thread.currentThread().getName());
    page.navigate("/products");
    page.getByPlaceholder("Search products").fill("lamp");
    assertThat(page.getByRole(AriaRole.LISTITEM)).hasCount(1);
  }
}
```

src/test/java/com/brainy/OrdersTest.java

```
package com.brainy;

import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class OrdersTest {
  @Test
  void pageOneShowsFiveOrders(Page page) {
    System.out.println("pageOneShowsFiveOrders on " + Thread.currentThread().getName());
    page.navigate("/orders");
    assertThat(page.getByRole(AriaRole.ROW)).hasCount(6);        // a header row and five orders
  }

  @Test
  void pageTwoIsTheLastPage(Page page) {
    System.out.println("pageTwoIsTheLastPage on " + Thread.currentThread().getName());
    page.navigate("/orders");
    page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Next")).click();
    assertThat(page.getByText("Page 2 of 2")).isVisible();
  }
}
```

Run it: `mvn test -Dtest=ProductsTest`

Lesson: Sharding across machines

TABLES

Table: junit 5

	JUnit 5
how	built in, threads in one JVM
turn it on	junit-platform.properties
how many	config.fixed.parallelism = 4
tests inside one file	in order, unless mode.default = concurrent
turn it off	parallel.enabled = false

Lesson: Workers: tests at the same time

Table: junit 5 with maven

	JUnit 5 with Maven
set retries	mvn test -Dsurefire.rerunFailingTestsCount=2
which try is this	not given, count it yourself
how it is reported	Flakes
fail the run on flaky tests	not built in

Lesson: Retries and flaky tests

Table: limit · default · java

Limit	Default	Java
one test	30 s	JUnit @Timeout(30)
one expect check	5 s	PlaywrightAssertions.setDefaultAssertionTimeout(5_000)
one action	no limit in Playwright Test, 30 s in the libraries	page.setDefaultTimeout()
one navigation	no limit in Playwright Test, 30 s in the libraries	page.setDefaultNavigationTimeout()
the whole run	no limit	not built in

Lesson: Timeouts

TIPS AND TRAPS

COMMON MISTAKE The Playwright docs advise against it: isolated tests are faster and easier to retry.

Lesson: Parallel tests must not share data

REMEMBER When a check times out, the fix is almost never a bigger number. Ask what the page was waiting for. A longer timeout is right only when the app really is that slow, such as a report that takes 20 s to build.

Lesson: Timeouts

STEP 5 Network and APIs

5.1 Network: watch, mock and change requests

Logging API traffic and reading one response, mocking the products API, a 500 and a broken connection, selling out the lamp in a real answer, recording and replaying a HAR file, and mocking the live stock WebSocket.

KEY IDEAS

- **Watch** See every call. Wait for the one you need.
- **Mock** Answer with your data. No server needed.
- **Change** Real answer, one field changed. The rest stays real.
- **Replay** Record once. Same data every run.
- **Remember** Mock what you do not own or cannot control. Keep a few tests against the real API, so the mocks do not drift away from it.

CODE

Listen to the traffic and wait for one response

src/test/java/com/brainy/WatchTest.java

```
package com.brainy;

import com.google.gson.JsonParser;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.Response;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;
import java.net.URI;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;
import static org.junit.jupiter.api.Assertions.assertTrue;

@UsePlaywright(ShopOptions.class)
public class WatchTest {
  @Test
  void theProductsComeFromTheApi(Page page) {
    page.onRequest(request → {
```

```
      if (request.url().contains("/api/")) System.out.println(">> " + request.method() + " " +
URI.create(request.url()).getPath());
    });
    page.onResponse(response -> {
      if (response.url().contains("/api/")) System.out.println("<< " + response.status() + " " +
URI.create(response.url()).getPath());
    });

    // run the navigation, and wait for the products call it causes
    Response response = page.waitForResponse("**/api/products", () -> page.navigate("/products"));

    assertTrue(response.ok());
    int count = JsonParser.parseString(response.text()).getAsJsonArray().size();
    System.out.println("the API sent " + count + " products");
    assertThat(page.getByRole(AriaRole.LISTITEM)).hasCount(count);
  }
}
```

Run it: `mvn test`

Lesson: Watch the traffic

Answer the products API with your own data

src/test/java/com/brainy/MockTest.java

```
package com.brainy;

import com.microsoft.playwright.Page;
import com.microsoft.playwright.Route;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class MockTest {
  static final String PRODUCTS = """
      [
        { "id": 7, "name": "Tea kettle", "price": 899, "stock": 4, "category": "Kitchen" },
        { "id": 8, "name": "Pencil box", "price": 150, "stock": 0, "category": "Stationery" }
      ]""";

  @Test
  void showsTheProductsTheApiSends(Page page) {
    page.route("**/api/products", route -> route.fulfill(new Route.FulfillOptions()
        .setContentType("application/json")
        .setBody(PRODUCTS)));

    page.navigate("/products");
    assertThat(page.getByRole(AriaRole.LISTITEM)).hasCount(2);
    assertThat(page.getByTestId("product-7")).containsText("₹899");
    assertThat(page.getByTestId("product-8").getByRole(AriaRole.BUTTON)).hasText("Out of stock");
  }
}
```

Run it: `mvn test`

Lesson: Mock an API

Make the API fail, and check the page copes

src/test/java/com/brainy/ErrorsTest.java

```
package com.brainy;

import com.microsoft.playwright.Page;
import com.microsoft.playwright.Route;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class ErrorsTest {
```

```
  static final String SORRY = "Sorry, we could not load the products. Please try again.";

  @Test
  void aServerErrorShowsAFriendlyMessage(Page page) {
    page.route("**/api/products", route -> route.fulfill(new Route.FulfillOptions()
        .setStatus(500).setContentType("application/json").setBody("{\"error\": \"Database is down\"}")));
    page.navigate("/products");
    assertThat(page.getByRole(AriaRole.ALERT)).hasText(SORRY);
  }

  @Test
  void aBrokenConnectionShowsTheSameMessage(Page page) {
    page.route("**/api/products", route -> route.abort());
    page.navigate("/products");
    assertThat(page.getByRole(AriaRole.ALERT)).hasText(SORRY);
  }
}
```

Run it: `mvn test`

Lesson: Errors that are hard to cause

Keep the real answer, change one field

src/test/java/com/brainy/SoldOutTest.java

```
package com.brainy;

import com.google.gson.JsonArray;
import com.google.gson.JsonElement;
import com.google.gson.JsonParser;
import com.microsoft.playwright.APIResponse;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.Route;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class SoldOutTest {
  @Test
  void theLampShowsAsSoldOut(Page page) {
    page.route("**/api/products", route -> {
      APIResponse response = route.fetch();                    // ask the real server
      JsonArray products = JsonParser.parseString(response.text()).getAsJsonArray();
      for (JsonElement product : products) {                   // then change one field
        if (product.getAsJsonObject().get("name").getAsString().equals("Desk lamp"))
          product.getAsJsonObject().addProperty("stock", 0);
      }
      route.fulfill(new Route.FulfillOptions().setResponse(response).setBody(products.toString()));
    });

    page.navigate("/products");
    assertThat(page.getByRole(AriaRole.LISTITEM)).hasCount(5);   // the other products are real
    assertThat(page.getByTestId("product-3").getByRole(AriaRole.BUTTON)).hasText("Out of stock");
  }
}
```

Run it: `mvn test`

Lesson: Change a real response

Record the API into a HAR file, then replay it

src/test/java/com/brainy/HarTest.java

```
package com.brainy;

import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.*;
import java.nio.file.Paths;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;
```

```java
@UsePlaywright(ShopOptions.class)
@TestMethodOrder(MethodOrderer.OrderAnnotation.class)
public class HarTest {
  @Test
  @Order(1)
  void recordTheProductsApi(Page page) {
    page.routeFromHAR(Paths.get("hars/products.har"), new
Page.RouteFromHAROptions().setUrl("**/api/products").setUpdate(true));
    page.navigate("/products");
    assertThat(page.getByRole(AriaRole.LISTITEM)).hasCount(5);
  }

  @Test
  @Order(2)
  void replayItFromTheFile(Page page) {
    // no update: answers come from the file
    page.routeFromHAR(Paths.get("hars/products.har"), new
Page.RouteFromHAROptions().setUrl("**/api/products"));
    page.navigate("/products");
    assertThat(page.getByRole(AriaRole.LISTITEM)).hasCount(5);
  }
}
```

Run it: `mvn test && ls -1 hars`

Lesson: Record and replay with HAR

Mock a WebSocket feed

src/test/java/com/brainy/LiveStockTest.java

```java
package com.brainy;

import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class LiveStockTest {
  @Test
  void showsTheLiveStockFromTheServer(Page page) {
    page.navigate("/widgets");
    assertThat(page.getByTestId("live-stock")).hasText("Desk lamp: 8 left");
  }

  @Test
  void aMockedFeedSaysTheLampSoldOut(Page page) {
    page.routeWebSocket("**/ws/stock", ws -> {               // no real server is contacted
      ws.onMessage(frame -> {
        if ("subscribe".equals(frame.text()))
          ws.send("{\"product\": \"Desk lamp\", \"left\": 0}");
      });
    });
    page.navigate("/widgets");
    assertThat(page.getByTestId("live-stock")).hasText("Desk lamp: sold out");
  }
}
```

Run it: `mvn test`

Lesson: WebSockets

TABLES

Table: to · java

To	Java
log every request	page.onRequest(...)
log every response	page.onResponse(...)
wait for one response	page.waitForResponse(..., () -> action)
wait for one request	page.waitForRequest(..., () -> action)
see failed requests	page.onRequestFailed(...)

Lesson: Watch the traffic

Table: step · what you write

Step	What you write
record	routeFromHAR(file, { url, update: true }) and run the test once
edit	change the JSON in the body file next to the HAR, if you want different data
replay	routeFromHAR(file, { url }) without update
refresh	run once more with update: true when the API changes

Lesson: Record and replay with HAR

TIPS AND TRAPS

REMEMBER The pattern in page.route is a glob: ** matches any part of the address, including slashes, and * matches any part between two slashes. A regular expression works too. Routes added with context.route catch requests from every page of the context, including new tabs.

Lesson: Mock an API

IN REAL TEAMS Routes can also make tests faster. page.route('**/*.{png,jpg,jpeg}', route => route.abort()) stops images from loading in tests that do not need them.

Lesson: Errors that are hard to cause

REMEMBER route.continue can change the request on its way out instead: a different header, method or body. For example, route.continue({ headers: { ...route.request().headers(), "x-test": "yes" } }).

Lesson: Change a real response

COMMON MISTAKE Replay matches the address and the method exactly, and for POST also the body. A request that matches nothing in the file is aborted, unless you pass notFound: fallback.

Lesson: Record and replay with HAR

5.2 API testing and signing in once

A full create, read, update and delete test of the people API with a validation error, a token sent in a header, a cart filled through the API and checked in the page, and a saved sign in reused by every test.

KEY IDEAS

- **API tests** No browser at all. Status, data, errors.
- **Tokens** Proof of who calls. Keep them in secrets.
- **Mix** API for set up. Page for the check.
- **Sign in once** Save the state. Every test starts signed in.
- **Remember** Use the page for what the user sees. Use the API for everything you only need to get ready.

CODE

Create, read, update and delete through the API

src/test/java/com/brainy/PeopleApiTest.java

```
package com.brainy;

import com.google.gson.JsonObject;
import com.google.gson.JsonParser;
import com.microsoft.playwright.APIRequestContext;
import com.microsoft.playwright.APIResponse;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.RequestOptions;
import org.junit.jupiter.api.Test;
import java.util.Map;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;
import static org.junit.jupiter.api.Assertions.assertEquals;

@UsePlaywright(ApiOptions.class)
public class PeopleApiTest {
  @Test
  void createReadUpdateAndDeleteAPerson(APIRequestContext request) {
    APIResponse created = request.post("/api/people",
        RequestOptions.create().setData(Map.of("name", "Kiran", "email", "kiran@shop.example")));
    assertEquals(201, created.status());
    int id = JsonParser.parseString(created.text()).getAsJsonObject().get("id").getAsInt();
    assertEquals("/api/people/" + id, created.headers().get("location"));

    APIResponse read = request.get("/api/people/" + id);
    assertThat(read).isOK();
    JsonObject kiran = JsonParser.parseString(read.text()).getAsJsonObject();
    assertEquals("Kiran", kiran.get("name").getAsString());
    assertEquals("kiran@shop.example", kiran.get("email").getAsString());

    APIResponse updated = request.patch("/api/people/" + id, RequestOptions.create().setData(Map.of("city",
"Pune")));
    assertEquals("Pune",
JsonParser.parseString(updated.text()).getAsJsonObject().get("city").getAsString());

    assertEquals(204, request.delete("/api/people/" + id).status());
    assertEquals(404, request.get("/api/people/" + id).status());
  }

  @Test
  void aBadEmailIsRefused(APIRequestContext request) {
    APIResponse response = request.post("/api/people",
        RequestOptions.create().setData(Map.of("name", "Nobody", "email", "not-an-email")));
    assertEquals(422, response.status());
    assertEquals("{\"error\":\"name and a valid email are required\"}", response.text());
  }
}
```

Run it: `mvn test`

Lesson: API tests without a browser

Sign in through the API and send the token

src/test/java/com/brainy/TokenTest.java

```
package com.brainy;

import com.google.gson.JsonObject;
import com.google.gson.JsonParser;
import com.microsoft.playwright.APIRequest;
import com.microsoft.playwright.APIRequestContext;
import com.microsoft.playwright.Playwright;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.RequestOptions;
import org.junit.jupiter.api.Test;
import java.util.Map;

import static org.junit.jupiter.api.Assertions.assertEquals;
```

```java
@UsePlaywright(ApiOptions.class)
public class TokenTest {
  @Test
  void theTokenOpensTheCart(APIRequestContext request, Playwright playwright) {
    assertEquals(401, request.get("/api/cart").status());      // nobody is signed in yet

    JsonObject login = JsonParser.parseString(request.post("/api/login",
        RequestOptions.create().setData(Map.of("email", "asha@shop.example", "password",
"s3cret"))).text()).getAsJsonObject();
    System.out.println("signed in as " + login.get("name").getAsString());

    // A fresh client that sends the token with every call, and has no cookies.
    APIRequestContext asha = playwright.request().newContext(new APIRequest.NewContextOptions()
        .setBaseURL("http://localhost:4173")
        .setExtraHTTPHeaders(Map.of("Authorization", "Bearer " + login.get("token").getAsString())));
    asha.post("/api/cart", RequestOptions.create().setData(Map.of("productId", 2, "qty", 2)));
    JsonObject cart = JsonParser.parseString(asha.get("/api/cart").text()).getAsJsonObject();
    System.out.println(cart);
    assertEquals(2, cart.get("count").getAsInt());
    assertEquals(398, cart.get("total").getAsInt());
    asha.dispose();
  }
}
```

Run it: `mvn test`

Lesson: Tokens and headers

Arrange through the API, check through the page

src/test/java/com/brainy/CartSetupTest.java

```java
package com.brainy;

import com.google.gson.JsonParser;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import com.microsoft.playwright.options.RequestOptions;
import org.junit.jupiter.api.Test;
import java.util.Map;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class CartSetupTest {
  @Test
  void aLampAddedThroughTheApiShowsInTheCartPage(Page page) {
    // Sign in through the API. page.request() shares cookies with the page.
    String token = JsonParser.parseString(page.request().post("/api/login",
        RequestOptions.create().setData(Map.of("email", "asha@shop.example", "password", "s3cret"))).text())
        .getAsJsonObject().get("token").getAsString();
    page.addInitScript("localStorage.setItem('token', '" + token + "')");    // the shop keeps its token
here

    page.request().post("/api/cart", RequestOptions.create().setData(Map.of("productId", 3)));   // arrange
the cart without clicking

    page.navigate("/cart");
    assertThat(page.getByText("Hello, Asha")).isVisible();
    assertThat(page.getByRole(AriaRole.ROW, new Page.GetByRoleOptions().setName("Desk
lamp"))).containsText("₹1,299");
    assertThat(page.getByTestId("cart-count")).hasText("1");
  }
}
```

Run it: `mvn test`

Lesson: API for set up, page for checks

Sign in once, save the state, start every test signed in

src/test/java/com/brainy/SignedInOptions.java

```java
package com.brainy;

import com.microsoft.playwright.Browser;
```

```java
import com.microsoft.playwright.junit.Options;
import com.microsoft.playwright.junit.OptionsFactory;
import java.nio.file.Paths;

// Every test that uses these options starts signed in as Asha.
public class SignedInOptions implements OptionsFactory {
  @Override
  public Options getOptions() {
    return new Options()
        .setHeadless(true)
        .setBaseUrl("http://localhost:4173")
        .setContextOptions(new
Browser.NewContextOptions().setStorageStatePath(Paths.get("playwright/.auth/asha.json")));
  }
}
```

src/test/java/com/brainy/SignedInTest.java

```java
package com.brainy;

import com.microsoft.playwright.*;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.Test;
import java.nio.file.Paths;
import java.util.regex.Pattern;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(SignedInOptions.class)
public class SignedInTest {
  @BeforeAll
  static void signInOnce() {
    // Sign in once, and save cookies and local storage for every test in the class.
    try (Playwright playwright = Playwright.create()) {
      BrowserContext context = playwright.chromium().launch()
          .newContext(new Browser.NewContextOptions().setBaseURL("http://localhost:4173"));
      Page page = context.newPage();
      page.navigate("/login");
      page.getByLabel("Email").fill("asha@shop.example");
      page.getByLabel("Password").fill("s3cret");
      page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Sign in")).click();
      assertThat(page.getByText("Hello, Asha")).isVisible();     // wait until signing in has finished
      context.storageState(new
BrowserContext.StorageStateOptions().setPath(Paths.get("playwright/.auth/asha.json")));
    }
  }

  @Test
  void theProductsPageGreetsAsha(Page page) {
    page.navigate("/products");                          // no sign in steps here
    assertThat(page.getByText("Hello, Asha")).isVisible();
  }

  @Test
  void theCartIsOpenToHer(Page page) {
    page.navigate("/cart");
    assertThat(page.getByRole(AriaRole.HEADING, new Page.GetByRoleOptions().setName("Your
cart"))).isVisible();
    assertThat(page).hasURL(Pattern.compile("/cart$"));  // not sent to /login
  }
}
```

Run it: `mvn test`

Lesson: Sign in once for the whole suite

TABLES

Table: java

	Java
the client	the APIRequestContext fixture, or playwright.request().newContext()
send JSON	request.post(url, RequestOptions.create().setData(...))
check status 200 to 299	assertThat(response).isOK()
read JSON	response.text() and a JSON library

Lesson: API tests without a browser

Table: need · approach

Need	Approach
all tests can share one account	one setup step and one state file, as above
tests change data of the account	one account per worker, named with the worker index
two roles, such as customer and admin	two state files, asha.json and ravi.json, and a project or fixture for each
the app keeps its session in sessionStorage	storageState does not save sessionStorage, so save it yourself and restore it with addInitScript

Lesson: Sign in once for the whole suite

TIPS AND TRAPS

COMMON MISTAKE Keep real passwords and tokens out of test code. Read them from environment variables, such as process.env.SHOP_PASSWORD, os.environ["SHOP_PASSWORD"] or System.getenv("SHOP_PASSWORD"), and set them as secrets in CI. The framework chapter shows a .env file for local runs.

Lesson: Tokens and headers

REMEMBER page.request (and context.request) share cookies with the browser context. A sign in through page.request signs in the page too, for apps that use cookies. The shop also keeps a token in local storage, which is why the example adds it with addInitScript.

Lesson: API for set up, page for checks

COMMON MISTAKE The state file holds a live session that anyone could use. Add playwright/.auth to .gitignore, and let each run create it fresh.

Lesson: Sign in once for the whole suite

STEP 6 Debugging and quality

6.1 Debugging failed tests: errors, traces, reports and the Inspector

A failed check read line by line in four languages, a failed click that leaves a screenshot, a video and a trace, a trace with named steps, verbose logs, and real screenshots of the trace viewer, the HTML report, UI mode and the Inspector.

KEY IDEAS

- **Error** Expected and received. Read it first.
- **Evidence** Screenshot, video, trace. Kept on failure.
- **Trace** Every action replayed. DOM, network, console.
- **Inspector** Run it yourself. One step at a time.
- **Remember** Read the error, then the trace, and only then run the test by hand. Most failures are solved at the first step.

CODE

A failed check, and what the runner prints

src/test/java/com/brainy/GreetingTest.java

```
package com.brainy;

import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class GreetingTest {
  @Test
  void greetsAshaAfterSignIn(Page page) {
    page.navigate("/login");
    page.getByLabel("Email").fill("asha@shop.example");
    page.getByLabel("Password").fill("s3cret");
    page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Sign in")).click();
    assertThat(page.locator("#hello")).hasText("Hello, Asha!");   // the page says Hello, Asha
  }
}
```

Run it: `mvn test`

Lesson: Read the error first

A screenshot, a video and a trace when a test fails

src/test/java/com/brainy/EvidenceOptions.java

```
package com.brainy;

import com.microsoft.playwright.Browser;
import com.microsoft.playwright.junit.Options;
import com.microsoft.playwright.junit.OptionsFactory;

import java.nio.file.Paths;

public class EvidenceOptions implements OptionsFactory {
  @Override
  public Options getOptions() {
    return new Options()
        .setHeadless(true)
        .setBaseUrl("http://localhost:4173")
        .setTrace(Options.Trace.RETAIN_ON_FAILURE)          // keep the trace only when a test fails
        .setOutputDir(Paths.get("test-results"))
        .setContextOptions(new Browser.NewContextOptions()
            .setRecordVideoDir(Paths.get("videos")));       // a video of every test
  }
}
```

src/test/java/com/brainy/CartTest.java

```
package com.brainy;

import com.microsoft.playwright.Locator;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(EvidenceOptions.class)
public class CartTest {
  @BeforeEach
  void signIn(Page page) {
    page.navigate("/login");
    page.getByLabel("Email").fill("asha@shop.example");
    page.getByLabel("Password").fill("s3cret");
```

```
    page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Sign in")).click();
    assertThat(page.getByRole(AriaRole.LISTITEM)).hasCount(5);
  }

  @Test
  void addTheLampToTheCart(Page page) {
    page.getByTestId("product-3").getByRole(AriaRole.BUTTON, new Locator.GetByRoleOptions().setName("Add to
cart")).click();
    assertThat(page.getByTestId("cart-count")).hasText("1");
  }

  @Test
  void addTheNotebookToTheCart(Page page) {
    // the Notebook is out of stock, so this button never appears
    page.getByTestId("product-4").getByRole(AriaRole.BUTTON, new Locator.GetByRoleOptions().setName("Add to
cart"))
        .click(new Locator.ClickOptions().setTimeout(3000));
  }
}
```

Run it: `mvn test ; find test-results videos -type f | sort`

Lesson: Screenshots, videos and traces on failure

A trace with named steps

src/test/java/com/brainy/MugTest.java

```
package com.brainy;

import com.microsoft.playwright.BrowserContext;
import com.microsoft.playwright.Locator;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.Tracing;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;

import java.nio.file.Paths;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class MugTest {
  @Test
  void buyAMug(BrowserContext context, Page page) {
    context.tracing().start(new Tracing.StartOptions().setScreenshots(true).setSnapshots(true));

    context.tracing().group("sign in");
    page.navigate("/login");
    page.getByLabel("Email").fill("asha@shop.example");
    page.getByLabel("Password").fill("s3cret");
    page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Sign in")).click();
    assertThat(page.locator("#hello")).hasText("Hello, Asha");
    context.tracing().groupEnd();

    context.tracing().group("add the mug");
    page.getByTestId("product-2").getByRole(AriaRole.BUTTON, new Locator.GetByRoleOptions().setName("Add to
cart")).click();
    assertThat(page.getByTestId("cart-count")).hasText("1");
    context.tracing().groupEnd();

    context.tracing().stop(new Tracing.StopOptions().setPath(Paths.get("traces/buy-a-mug.zip")));
  }
}
```

Run it: `mvn test && ls traces`

Lesson: The trace viewer

Every Playwright call, written to the terminal

src/test/java/com/brainy/LampTest.java

```
package com.brainy;

import com.microsoft.playwright.Page;
```

```java
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class LampTest {
  @Test
  void findTheLamp(Page page) {
    page.navigate("/products");
    page.getByPlaceholder("Search products").fill("lamp");
    assertThat(page.getByRole(AriaRole.LISTITEM)).hasCount(1);
  }
}
```

Run it: `DEBUG=pw:api mvn test`

Lesson: Verbose logs

TABLES

Table: part · what it tells you

Part	What it tells you
the locator	which element the check or action looked for
expected and received	what the test wanted and what the page had, side by side
the timeout	how long Playwright kept trying before it gave up
the call log	each try: did the locator find an element, was it visible, enabled and stable, what did it contain

Lesson: Read the error first

Table: the error says · it usually means

The error says	It usually means
received a different text or count	the page changed, or the expected value in the test is out of date
element(s) not found, or waiting for locator	the locator is wrong, a step before it did not happen, or the page did not load
strict mode violation, resolved to 2 elements	the locator is too loose, so narrow it with a role, a name or a parent
element is not enabled, or is not visible	the app has not finished something, such as loading data or validating a form
Test timeout of 30000ms exceeded	one call waited for something that never happened, and the call log names it

Lesson: Read the error first

Table: setting · junit fixture

Setting	JUnit fixture
screenshot	not built in, call page.screenshot() yourself
video	setRecordVideoDir(...), every test
trace	setTrace(Options.Trace.RETAIN_ON_FAILURE)
where	setOutputDir(...)
other values	ON, OFF

Lesson: Screenshots, videos and traces on failure

Table: open a trace · command

Open a trace	Command
Java	mvn exec:java -e -D exec.mainClass=com.microsoft.playwright.CLI -D exec.args="show-trace trace.zip"
any language, no install	drag the file onto trace.playwright.dev, which reads it in your browser

Lesson: The trace viewer

Table: java

	Java
open the Inspector	PWDEBUG=1 mvn test
one test only	PWDEBUG=1 mvn test -Dtest=CartTest#addTheNotebookToTheCart
stop at a line	page.pause()
see the source	PLAYWRIGHT_JAVA_SRC=src/test/java
watch slowly	setLaunchOptions(...setSlowMo(500))

Lesson: Step through a test with the Inspector

Table: variable · what it prints

Variable	What it prints
DEBUG=pw:api	every Playwright call and the steps inside it
DEBUG=pw:browser	what the browser process prints, useful when it will not start
DEBUG=pw:protocol	every message between Playwright and the browser, very long
PWDEBUG=console	a playwright object in the browser DevTools, to try locators in the console

Lesson: Verbose logs

TIPS AND TRAPS

REMEMBER Read the received value carefully. A missing full stop, an extra space or a different currency sign is the most common cause of a red check, and the error shows it to you directly.
Lesson: Read the error first

IN REAL TEAMS Teams often use trace: 'on-first-retry' in CI. A first run that fails is retried with tracing on, so the trace is recorded only for tests that failed at least once.
Lesson: Screenshots, videos and traces on failure

REMEMBER Inside the trace viewer, click an action and then use the Before and After tabs to compare the page. The Pick locator button lets you try a new locator on the saved page.
Lesson: The trace viewer

COMMON MISTAKE Remove page.pause() before you commit. It needs a headed browser and is only useful while you debug.
Lesson: Step through a test with the Inspector

REMEMBER Browser console messages and page errors are a different stream. Collect them with page.on("console") and page.on("pageerror"), as the fixtures chapter does, or read them in the Console tab of the trace.
Lesson: Verbose logs

6.2 Visual, accessibility and time checks

A screenshot comparison that catches a style bug with a real diff picture, aria snapshots of the menu and a product card, an axe scan that finds a missing alt text and low contrast, and a sale date and an idle sign out tested with a fake clock.

KEY IDEAS

- **Screenshot** How it looks. Baseline and diff.
- **Aria snapshot** How it reads. Every language.
- **axe scan** Who can use it. WCAG rules.
- **Fake clock** Any date, any delay. No waiting.

- **Remember** Screenshots check pixels, aria snapshots check structure, axe checks access, and the clock checks time. Use each for the mistakes it is good at.

CODE

Check the structure of the page with an aria snapshot

src/test/java/com/brainy/StructureTest.java

```
package com.brainy;

import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class StructureTest {
  @Test
  void theMainMenuHasFourLinks(Page page) {
    page.navigate("/products");
    assertThat(page.getByRole(AriaRole.NAVIGATION, new
Page.GetByRoleOptions().setName("Main"))).matchesAriaSnapshot("""
        - navigation "Main":
          - link "Products"
          - link "Feedback"
          - link "Widgets"
          - link "Orders"
        """);
  }

  @Test
  void theNotebookCardSaysItIsOutOfStock(Page page) {
    page.navigate("/products");
    System.out.println(page.getByTestId("product-4").ariaSnapshot());   // print it once, then paste it
below
    assertThat(page.getByTestId("product-4")).matchesAriaSnapshot("""
        - listitem:
          - heading "Notebook" [level=3]
          - text: Stationery ₹60
          - button "Out of stock" [disabled]
        """);
  }
}
```

Run it: `mvn test`

Lesson: Aria snapshots

Scan a page for accessibility problems with axe

src/test/java/com/brainy/OffersAccessibilityTest.java

```
package com.brainy;

import com.deque.html.axecore.playwright.AxeBuilder;
import com.deque.html.axecore.results.AxeResults;
import com.deque.html.axecore.results.Rule;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import org.junit.jupiter.api.Test;

import java.util.List;

import static org.junit.jupiter.api.Assertions.assertEquals;

@UsePlaywright(ShopOptions.class)
public class OffersAccessibilityTest {
  @Test
  void theOffersPageHasNoAccessibilityProblems(Page page) {
    page.navigate("/a11y");
    AxeResults results = new AxeBuilder(page)
        .withTags(List.of("wcag2a", "wcag2aa"))      // the WCAG 2 A and AA rules
```

```
        .analyze();
    for (Rule v : results.getViolations()) {
      System.out.println(v.getId() + " (" + v.getImpact() + "): " + v.getNodes().size() + " element(s). " +
v.getHelp());
    }
    assertEquals(List.of(), results.getViolations().stream().map(Rule::getId).toList());
  }
}
```

Run it: `mvn test`

Lesson: Accessibility scans with axe

Control dates and timers with a fake clock

src/test/java/com/brainy/SaleClockTest.java

```
package com.brainy;

import com.microsoft.playwright.Clock;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class SaleClockTest {
  @Test
  void theSaleIsNotLiveTheDayBefore(Page page) {
    page.clock().setFixedTime("2026-12-24T12:00:00Z");
    page.navigate("/clock");
    assertThat(page.getByRole(AriaRole.STATUS)).hasText("The Christmas sale is coming soon");
  }

  @Test
  void theSaleGoesLiveOnChristmasDay(Page page) {
    page.clock().setFixedTime("2026-12-25T12:00:00Z");
    page.navigate("/clock");
    assertThat(page.getByRole(AriaRole.STATUS)).hasText("The Christmas sale is live");
  }

  @Test
  void anIdleUserIsSignedOutAfterFiveMinutes(Page page) {
    page.clock().install(new Clock.InstallOptions().setTime("2026-12-24T10:00:00Z"));
    page.navigate("/clock");
    page.clock().runFor("05:00");                          // 5 minutes of timers, in a moment
    assertThat(page.getByRole(AriaRole.ALERT)).hasText("You were signed out");
  }
}
```

Run it: `mvn test`

Lesson: Fake time with page.clock

TABLES

Table: option · what it does

Option	What it does
--update-snapshots	saves new baselines instead of comparing, after a change you meant to make
maxDiffPixels, maxDiffPixelRatio	how many pixels, or what share of them, may differ
threshold	how different one pixel may be before it counts, from 0 to 1, default 0.2
mask: [locator]	covers parts that change every run, such as a date or an advert, with a box
animations: "disabled"	stops CSS animations so the picture is the same every time (the default)
stylePath	adds a style sheet only while taking the picture, for example to hide a clock

Lesson: Visual comparisons

Table: java

	Java
print a snapshot	locator.ariaSnapshot()
check a snapshot	matchesAriaSnapshot("""...""")
keep it in a file	not built in

Lesson: Aria snapshots

Table: what axe found · why it matters · the fix

What axe found	Why it matters	The fix
image-alt: the lamp image has no alt text	a screen reader cannot say what the picture shows	add alt="Desk lamp"
color-contrast: light blue text on a light background	people with low vision cannot read it	use a darker colour, with a contrast ratio of at least 4.5 to 1

Lesson: Accessibility scans with axe

Table: package · language · made by

Package	Language	Made by
@axe-core/playwright	JavaScript and TypeScript	Deque
com.deque.html.axe-core:playwright	Java	Deque
axe-playwright-python	Python	community

Lesson: Accessibility scans with axe

Table: method · what it does

Method	What it does
setFixedTime(time)	Date.now() always returns this time. Timers still run normally.
install({ time })	replaces Date and every timer. Call it before the page loads.
runFor("05:00")	moves forward, firing every timer that falls due on the way
fastForward("05:00")	jumps forward and fires due timers at most once, like a laptop waking from sleep
pauseAt(time), resume()	stops time at a moment to check the page, then lets it run again
setSystemTime(time)	changes the date without firing timers, like a user changing the system clock

Lesson: Fake time with page.clock

TIPS AND TRAPS

IN REAL TEAMS Baselines depend on the operating system and the browser, which is why the file name ends in chromium-linux. Make them on the same system that runs the tests, usually the CI image or the Playwright Docker image, and commit them to the repository.

Lesson: Visual comparisons

COMMON MISTAKE toHaveScreenshot is part of Playwright Test. Python and Java have page.screenshot() to save a picture, but no comparison built in. Teams add a library for it or use a visual testing service. The aria snapshot below works in every language.

Lesson: Visual comparisons

REMEMBER Codegen can record aria snapshots too: use the Assert snapshot button in the toolbar while recording, and the snapshot of the element you click is added to the test.

Lesson: Aria snapshots

COMMON MISTAKE An automatic scan finds only part of the problems. It cannot tell whether alt text is useful or whether the tab order makes sense. Keep a few manual checks with a keyboard and a screen reader, and use the scan to stop the easy mistakes from coming back.

Lesson: Accessibility scans with axe

IN REAL TEAMS An old page may have many known problems. Scan with .exclude() for the parts you will fix later, or compare the list of violations with a saved list, so new problems still fail the test.

Lesson: Accessibility scans with axe

REMEMBER Use setFixedTime when the page only reads the date, and install with runFor when it uses setTimeout or setInterval. Either way the test runs in milliseconds and gives the same result on every day of the year.

Lesson: Fake time with page.clock

STEP 7 Framework design

7.1 Page objects and components

A LoginPage, a ProductsPage built from Header and ProductCard components, and fixtures that hand out signed in page objects, each run for real in four languages.

KEY IDEAS

- **Page object** One class per screen. Locators in one place.
- **Component** Parts that repeat. Root locator first.
- **Fixtures** Hand them out. No set up in tests.
- **Checks** Stay in the test. Easy to read.
- **Remember** The test says what, the page object says how. A change to the screen is fixed in one class.

CODE

Step 1: one page object for the sign in page

src/test/java/com/brainy/pages/LoginPage.java

```
package com.brainy.pages;

import com.microsoft.playwright.Locator;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.options.AriaRole;

// Everything a test needs to know about the sign in page, in one place
public class LoginPage {
  private final Page page;
  public final Locator email;
  public final Locator password;
  public final Locator signInButton;
  public final Locator error;

  public LoginPage(Page page) {
    this.page = page;
    this.email = page.getByLabel("Email");
    this.password = page.getByLabel("Password");
    this.signInButton = page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Sign in"));
    this.error = page.getByRole(AriaRole.ALERT);
  }

  public void navigate() {
    page.navigate("/login");
  }

  public void signIn(String email, String password) {
    this.email.fill(email);
    this.password.fill(password);
```

```
    signInButton.click();
  }
}
```

src/test/java/com/brainy/LoginTest.java

```
package com.brainy;

import com.brainy.pages.LoginPage;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class LoginTest {
  @Test
  void ashaCanSignIn(Page page) {
    LoginPage loginPage = new LoginPage(page);
    loginPage.navigate();
    loginPage.signIn("asha@shop.example", "s3cret");
    assertThat(page.getByText("Hello, Asha")).isVisible();
  }

  @Test
  void aWrongPasswordIsRefused(Page page) {
    LoginPage loginPage = new LoginPage(page);
    loginPage.navigate();
    loginPage.signIn("asha@shop.example", "wrong");
    assertThat(loginPage.error).hasText("Wrong email or password");
  }
}
```

Run it: `mvn test`

Lesson: Why page objects

Step 2: components for parts that repeat

src/test/java/com/brainy/components/ProductCard.java

```
package com.brainy.components;

import com.microsoft.playwright.Locator;
import com.microsoft.playwright.options.AriaRole;

// One product card. Every locator starts from the card, so it never finds another product's button.
public class ProductCard {
  public final Locator root;
  public final Locator price;
  public final Locator button;

  public ProductCard(Locator root) {
    this.root = root;
    this.price = root.locator(".price");
    this.button = root.getByRole(AriaRole.BUTTON);
  }

  public void addToCart() {
    root.getByRole(AriaRole.BUTTON, new Locator.GetByRoleOptions().setName("Add to cart")).click();
  }
}
```

src/test/java/com/brainy/components/Header.java

```
package com.brainy.components;

import com.microsoft.playwright.Locator;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.options.AriaRole;

// The header is on every page, so it is a component, not a page
public class Header {
  public final Locator greeting;
  public final Locator cartCount;
  public final Locator signOut;
```

```
  public Header(Page page) {
    Locator root = page.getByRole(AriaRole.BANNER);
    this.greeting = root.locator("#hello");
    this.cartCount = root.getByTestId("cart-count");
    this.signOut = root.getByRole(AriaRole.LINK, new Locator.GetByRoleOptions().setName("Sign out"));
  }
}
```

src/test/java/com/brainy/pages/ProductsPage.java

```
package com.brainy.pages;

import com.brainy.components.Header;
import com.brainy.components.ProductCard;
import com.microsoft.playwright.Locator;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.options.AriaRole;

public class ProductsPage {
  private final Page page;
  public final Header header;
  public final Locator search;
  public final Locator cards;

  public ProductsPage(Page page) {
    this.page = page;
    this.header = new Header(page);
    this.search = page.getByPlaceholder("Search products");
    this.cards = page.getByRole(AriaRole.LISTITEM);
  }

  public void navigate() {
    page.navigate("/products");
  }

  public ProductCard card(String name) {
    return new ProductCard(cards.filter(new Locator.FilterOptions()
        .setHas(page.getByRole(AriaRole.HEADING, new Page.GetByRoleOptions().setName(name)))));
  }
}
```

src/test/java/com/brainy/CartTest.java

```
package com.brainy;

import com.brainy.components.ProductCard;
import com.brainy.pages.LoginPage;
import com.brainy.pages.ProductsPage;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@UsePlaywright(ShopOptions.class)
public class CartTest {
  @Test
  void ashaAddsTheLampAndTheMug(Page page) {
    LoginPage loginPage = new LoginPage(page);
    loginPage.navigate();
    loginPage.signIn("asha@shop.example", "s3cret");

    ProductsPage products = new ProductsPage(page);
    assertThat(products.header.greeting).hasText("Hello, Asha");
    products.card("Desk lamp").addToCart();
    assertThat(products.header.cartCount).hasText("1");
    products.card("Coffee mug").addToCart();
    assertThat(products.header.cartCount).hasText("2");
  }

  @Test
  void theNotebookIsOutOfStock(Page page) {
    ProductsPage products = new ProductsPage(page);
    products.navigate();
```

```
    ProductCard notebook = products.card("Notebook");
    assertThat(notebook.price).hasText("₹60");
    assertThat(notebook.button).hasText("Out of stock");
    assertThat(notebook.button).isDisabled();
  }
}
```

Run it: `mvn test`

Lesson: Components for repeated parts

Step 3: page objects as fixtures

src/test/java/com/brainy/ShopTest.java

```
package com.brainy;

import com.brainy.pages.LoginPage;
import com.brainy.pages.ProductsPage;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import org.junit.jupiter.api.BeforeEach;

// Every test class that extends ShopTest gets signed in page objects
@UsePlaywright(ShopOptions.class)
public abstract class ShopTest {
  protected LoginPage loginPage;
  protected ProductsPage productsPage;

  @BeforeEach
  void createPages(Page page) {
    loginPage = new LoginPage(page);
    productsPage = new ProductsPage(page);
    loginPage.navigate();
    loginPage.signIn("asha@shop.example", "s3cret");
  }
}
```

src/test/java/com/brainy/MugTest.java

```
package com.brainy;

import com.brainy.components.ProductCard;
import org.junit.jupiter.api.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

public class MugTest extends ShopTest {
  @Test
  void theMugCosts199RupeesAndCanBeAdded() {
    ProductCard mug = productsPage.card("Coffee mug");
    assertThat(mug.price).hasText("₹199");
    mug.addToCart();
    assertThat(productsPage.header.cartCount).hasText("1");
  }

  @Test
  void theSearchFindsOneLamp() {
    productsPage.search.fill("lamp");
    assertThat(productsPage.cards).hasCount(1);
  }
}
```

Run it: `mvn test`

Lesson: Handing out page objects

TABLES

Table: without page objects · with page objects

Without page objects	With page objects
locators copied into every test	each locator written once
a UI change breaks many files	a UI change is fixed in one class
tests read as clicks and fills	tests read as steps: signIn, addToCart
new team members copy old tests	new tests reuse a small, named set of methods

Lesson: Why page objects

Table: junit 5

	JUnit 5
where page objects are created	a base class with @BeforeEach
how a test gets one	a field of the base class
only built when used	no, for every test in the class

Lesson: Handing out page objects

Table: rule · why

Rule	Why
one class per screen or component	easy to find, small enough to read
locators by role, label or test id	they survive style changes, as the locators chapter showed
methods are user actions: signIn, addToCart	tests read like steps, not clicks
checks live in tests	the test shows what it proves
no sleeps and no waits for timeouts	Playwright actions and checks already wait
no test data inside the class	pass it in, so one page object serves many tests
a method returns the next page object when the screen changes	signIn() can return ProductsPage, so the flow is clear

Lesson: Rules teams follow

TIPS AND TRAPS

REMEMBER Return locators, or small components, and leave the checks in the test. A test that says expect(notebook.button).toBeDisabled() shows what it proves. A page object method called checkNotebook() hides it.

Lesson: Components for repeated parts

IN REAL TEAMS Large suites sometimes add one more layer of business steps, such as checkout(user, items), built from page objects. Add it when you notice the same five page object calls in many tests, not before.

Lesson: Rules teams follow

7.2 Framework design, step by step

The same framework built and run in TypeScript, JavaScript, Python and Java: the runner finds the tests, the full suite passes, the smoke set runs alone, and every file is shown with why it is there.

KEY IDEAS

- **Settings** From the environment. No secrets in Git.
- **Layers** One job each. Data, API, pages.
- **Fixtures** Fresh session per test. Safe in parallel.
- **Tests** By feature, tagged. Steps and checks only.

- **Remember** Framework code and tests live apart. Tests say what, the framework says how, and the environment says where.

CODE

Step 1: the project, its settings and its runner config

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.brainy</groupId>
  <artifactId>shop-tests</artifactId>
  <version>1.0.0</version>

  <properties>
    <maven.compiler.release>21</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <suite>testng.xml</suite>                <!-- mvn test -Dsuite=testng-smoke.xml runs another suite -->
  </properties>

  <dependencies>
    <dependency>
      <groupId>com.microsoft.playwright</groupId>
      <artifactId>playwright</artifactId>
      <version>1.63.0</version>
    </dependency>
    <dependency>
      <groupId>org.testng</groupId>
      <artifactId>testng</artifactId>
      <version>7.11.0</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>3.5.4</version>
        <configuration>
          <suiteXmlFiles>
            <suiteXmlFile>${suite}</suiteXmlFile>
          </suiteXmlFiles>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

testng.xml

```
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<!-- Every test, two classes at a time. Each class gets its own browser on its own thread. -->
<suite name="Brainy Shop" parallel="classes" thread-count="2">
  <test name="All tests">
    <packages>
      <package name="com.brainy.shop.tests.*"/>
    </packages>
  </test>
</suite>
```

testng-smoke.xml

```
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<!-- Only the tests in the smoke group, for every commit -->
<suite name="Brainy Shop smoke" parallel="classes" thread-count="2">
  <test name="Smoke">
    <groups>
      <run>
        <include name="smoke"/>
      </run>
    </groups>
    <packages>
```

```
      <package name="com.brainy.shop.tests.*"/>
    </packages>
  </test>
</suite>
```

src/test/resources/config.properties

```
# Default settings. A -D option or an environment variable overrides each one.
baseUrl=http://localhost:4173
headless=true
```

src/main/java/com/brainy/shop/config/Config.java

```
package com.brainy.shop.config;

import java.io.IOException;
import java.io.InputStream;
import java.util.Properties;

// Settings that change between machines (a laptop, CI, staging) live here and nowhere else.
// Order: -DbaseUrl=..., then the BASE_URL environment variable, then config.properties.
public final class Config {
  private static final Properties FILE = new Properties();

  static {
    try (InputStream in = Config.class.getResourceAsStream("/config.properties")) {
      if (in != null) FILE.load(in);
    } catch (IOException e) {
      throw new IllegalStateException("Cannot read config.properties", e);
    }
  }

  private Config() {}

  public static String get(String key, String envName) {
    String value = System.getProperty(key);
    if (value == null) value = System.getenv(envName);
    if (value == null) value = FILE.getProperty(key);
    return value;
  }

  public static String baseUrl() {
    return get("baseUrl", "BASE_URL");
  }

  public static boolean headless() {
    return Boolean.parseBoolean(get("headless", "HEADLESS"));
  }
}
```

Run it: `mvn -q test-compile && find src/test -name "*Test.java" | sort`

Lesson: Step 1: project, settings and runner config

Step 2: data, API helpers, components, page objects and fixtures

src/main/java/com/brainy/shop/data/User.java

```
package com.brainy.shop.data;

public record User(String email, String password, String name) {
  public User withPassword(String other) {
    return new User(email, other, name);
  }
}
```

src/main/java/com/brainy/shop/data/Users.java

```
package com.brainy.shop.data;

// Test accounts. Real projects read passwords from CI secrets, never from the repository.
public final class Users {
  public static final User ASHA = new User("asha@shop.example", secret("ASHA_PASSWORD", "s3cret"), "Asha");
  public static final User RAVI = new User("ravi@shop.example", secret("RAVI_PASSWORD", "admin123"),
"Ravi");

  private Users() {}
```

```
  private static String secret(String name, String fallback) {
    String value = System.getenv(name);
    return value == null || value.isEmpty() ? fallback : value;
  }
}
```

src/main/java/com/brainy/shop/data/Product.java

```
package com.brainy.shop.data;

// The products the tests use, with the ids the API expects
public enum Product {
  GEL_PEN(1, "Gel pen", "₹20"),
  COFFEE_MUG(2, "Coffee mug", "₹199"),
  DESK_LAMP(3, "Desk lamp", "₹1,299"),
  NOTEBOOK(4, "Notebook", "₹60"),
  WATER_BOTTLE(5, "Water bottle", "₹349");

  public final int id;
  public final String label;
  public final String price;

  Product(int id, String label, String price) {
    this.id = id;
    this.label = label;
    this.price = price;
  }
}
```

src/main/java/com/brainy/shop/api/ShopApi.java

```
package com.brainy.shop.api;

import com.brainy.shop.data.User;
import com.microsoft.playwright.APIRequestContext;
import com.microsoft.playwright.APIResponse;
import com.microsoft.playwright.options.RequestOptions;

import java.util.Map;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

// Fast set up through the API, so UI tests only click what they are testing
public class ShopApi {
  private final APIRequestContext request;

  public ShopApi(APIRequestContext request) {
    this.request = request;
  }

  public String signIn(User user) {
    APIResponse response = request.post("/api/login",
        RequestOptions.create().setData(Map.of("email", user.email(), "password", user.password())));
    if (!response.ok()) throw new IllegalStateException("Sign in failed for " + user.email() + ": " +
response.status());
    Matcher token = Pattern.compile("\"token\":\"(\\w+)\"").matcher(response.text());
    if (!token.find()) throw new IllegalStateException("No token in " + response.text());
    return token.group(1);
  }

  public void addToCart(String token, int productId, int qty) {
    APIResponse response = request.post("/api/cart", RequestOptions.create()
        .setData(Map.of("productId", productId, "qty", qty))
        .setHeader("Authorization", "Bearer " + token));
    if (!response.ok()) throw new IllegalStateException("Could not add product " + productId + ": " +
response.status());
  }
}
```

src/main/java/com/brainy/shop/components/Header.java

```
package com.brainy.shop.components;

import com.microsoft.playwright.Locator;
```

```java
import com.microsoft.playwright.Page;
import com.microsoft.playwright.options.AriaRole;

// The header is on every page, so it is a component that every page object shares
public class Header {
  public final Locator greeting;
  public final Locator cartCount;
  public final Locator signOut;

  public Header(Page page) {
    Locator root = page.getByRole(AriaRole.BANNER);
    this.greeting = root.locator("#hello");
    this.cartCount = root.getByTestId("cart-count");
    this.signOut = root.getByRole(AriaRole.LINK, new Locator.GetByRoleOptions().setName("Sign out"));
  }
}
```

src/main/java/com/brainy/shop/components/ProductCard.java

```java
package com.brainy.shop.components;

import com.microsoft.playwright.Locator;
import com.microsoft.playwright.options.AriaRole;

// One product card. Every locator starts from the card, so it never finds another product's button.
public class ProductCard {
  public final Locator root;
  public final Locator price;
  public final Locator button;

  public ProductCard(Locator root) {
    this.root = root;
    this.price = root.locator(".price");
    this.button = root.getByRole(AriaRole.BUTTON);
  }

  public void addToCart() {
    root.getByRole(AriaRole.BUTTON, new Locator.GetByRoleOptions().setName("Add to cart")).click();
  }
}
```

src/main/java/com/brainy/shop/pages/BasePage.java

```java
package com.brainy.shop.pages;

import com.brainy.shop.components.Header;
import com.microsoft.playwright.Page;

// What every page shares: the browser page, its address and the header
public abstract class BasePage {
  protected final Page page;
  private final String path;
  public final Header header;

  protected BasePage(Page page, String path) {
    this.page = page;
    this.path = path;
    this.header = new Header(page);
  }

  public void open() {
    page.navigate(path);
  }
}
```

src/main/java/com/brainy/shop/pages/LoginPage.java

```java
package com.brainy.shop.pages;

import com.brainy.shop.data.User;
import com.microsoft.playwright.Locator;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.options.AriaRole;

public class LoginPage extends BasePage {
```

```
  public final Locator email;
  public final Locator password;
  public final Locator signInButton;
  public final Locator error;

  public LoginPage(Page page) {
    super(page, "/login");
    this.email = page.getByLabel("Email");
    this.password = page.getByLabel("Password");
    this.signInButton = page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Sign in"));
    this.error = page.getByRole(AriaRole.ALERT);
  }

  public void signIn(User user) {
    email.fill(user.email());
    password.fill(user.password());
    signInButton.click();
  }
}
```

src/main/java/com/brainy/shop/pages/ProductsPage.java

```
package com.brainy.shop.pages;

import com.brainy.shop.components.ProductCard;
import com.microsoft.playwright.Locator;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.options.AriaRole;

public class ProductsPage extends BasePage {
  public final Locator search;
  public final Locator cards;
  public final Locator noResults;

  public ProductsPage(Page page) {
    super(page, "/products");
    this.search = page.getByPlaceholder("Search products");
    this.cards = page.getByRole(AriaRole.LIST, new
Page.GetByRoleOptions().setName("Products")).getByRole(AriaRole.LISTITEM);
    this.noResults = page.getByText("No products match your search");
  }

  public ProductCard card(String name) {
    return new ProductCard(cards.filter(new Locator.FilterOptions()
        .setHas(page.getByRole(AriaRole.HEADING, new Page.GetByRoleOptions().setName(name)))));
  }

  public void searchFor(String text) {
    search.fill(text);
  }
}
```

src/main/java/com/brainy/shop/pages/CartPage.java

```
package com.brainy.shop.pages;

import com.microsoft.playwright.Locator;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.options.AriaRole;

public class CartPage extends BasePage {
  public final Locator items;
  public final Locator total;
  public final Locator placeOrderButton;
  public final Locator status;

  public CartPage(Page page) {
    super(page, "/cart");
    this.items = page.getByRole(AriaRole.TABLE, new Page.GetByRoleOptions().setName("Cart
items")).locator("tbody tr");
    this.total = page.getByTestId("total");
    this.placeOrderButton = page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Place
order"));
    this.status = page.getByRole(AriaRole.STATUS);
  }
```

```
  public void placeOrder() {
    page.onceDialog(dialog -> dialog.accept());   // the shop asks to confirm the order
    placeOrderButton.click();
  }
}
```

src/test/java/com/brainy/shop/base/BaseTest.java

```
package com.brainy.shop.base;

import com.brainy.shop.api.ShopApi;
import com.brainy.shop.config.Config;
import com.brainy.shop.data.User;
import com.brainy.shop.data.Users;
import com.brainy.shop.pages.CartPage;
import com.brainy.shop.pages.LoginPage;
import com.brainy.shop.pages.ProductsPage;
import com.microsoft.playwright.APIRequest;
import com.microsoft.playwright.APIRequestContext;
import com.microsoft.playwright.Browser;
import com.microsoft.playwright.BrowserContext;
import com.microsoft.playwright.BrowserType;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.Playwright;
import com.microsoft.playwright.Tracing;
import org.testng.ITestResult;
import org.testng.annotations.AfterClass;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.BeforeMethod;

import java.nio.file.Paths;

// Every test class extends BaseTest. TestNG runs classes in parallel, and each class has its own
// Playwright and browser, so nothing is shared between threads.
public abstract class BaseTest {
  private Playwright playwright;
  private Browser browser;
  private APIRequestContext request;
  private BrowserContext context;
  protected Page page;
  protected ShopApi api;
  protected LoginPage loginPage;
  protected ProductsPage productsPage;
  protected CartPage cartPage;

  @BeforeClass(alwaysRun = true)        // alwaysRun: also when a group such as smoke is chosen
  public void launchBrowser() {
    playwright = Playwright.create();
    browser = playwright.chromium().launch(new BrowserType.LaunchOptions().setHeadless(Config.headless()));
    request = playwright.request().newContext(new
APIRequest.NewContextOptions().setBaseURL(Config.baseUrl()));
    api = new ShopApi(request);
  }

  @BeforeMethod(alwaysRun = true)
  public void openPage() {
    context = browser.newContext(new Browser.NewContextOptions().setBaseURL(Config.baseUrl()));
    context.tracing().start(new Tracing.StartOptions().setScreenshots(true).setSnapshots(true));
    page = context.newPage();
    loginPage = new LoginPage(page);
    productsPage = new ProductsPage(page);
    cartPage = new CartPage(page);
  }

  // A fresh session made through the API: fast, and no two tests share a cart
  protected String signInThroughApi(User user) {
    String token = api.signIn(user);
    context.addInitScript("window.localStorage.setItem('token', '" + token + "')");
    return token;
  }

  protected String signInThroughApi() {
```

```
    return signInThroughApi(Users.ASHA);
  }

  // Report listeners call this when a test fails. TestNG calls them before closePage(), so the page is
still open.
  public byte[] screenshot() {
    return page.screenshot();
  }

  @AfterMethod(alwaysRun = true)
  public void closePage(ITestResult result) {
    if (result.isSuccess()) {
      context.tracing().stop();
    } else {                                        // keep a trace only for a failed test
      String name = getClass().getSimpleName() + "." + result.getMethod().getMethodName();
      context.tracing().stop(new Tracing.StopOptions().setPath(Paths.get("traces", name + ".zip")));
    }
    context.close();
  }

  @AfterClass(alwaysRun = true)
  public void closeBrowser() {
    request.dispose();
    playwright.close();
  }
}
```

Run it: `mvn test`

Lesson: Step 2: data, API helpers, page objects and fixtures

Step 3: tests by feature, tagged so a smaller set can run

src/test/java/com/brainy/shop/tests/smoke/SignInTest.java

```
package com.brainy.shop.tests.smoke;

import com.brainy.shop.base.BaseTest;
import com.brainy.shop.data.Users;
import org.testng.annotations.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@Test(groups = "smoke")
public class SignInTest extends BaseTest {
  @Test
  public void ashaSignsInWithTheRightPassword() {
    loginPage.open();
    loginPage.signIn(Users.ASHA);
    assertThat(productsPage.header.greeting).hasText("Hello, Asha");
  }

  @Test
  public void aWrongPasswordShowsAnError() {
    loginPage.open();
    loginPage.signIn(Users.ASHA.withPassword("wrong"));
    assertThat(loginPage.error).hasText("Wrong email or password");
  }
}
```

src/test/java/com/brainy/shop/tests/products/SearchTest.java

```
package com.brainy.shop.tests.products;

import com.brainy.shop.base.BaseTest;
import com.brainy.shop.data.Product;
import org.testng.annotations.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

public class SearchTest extends BaseTest {
  @Test
  public void findsTheLampByName() {
    productsPage.open();
    productsPage.searchFor("lamp");
    assertThat(productsPage.cards).hasCount(1);
```

```
    assertThat(productsPage.card(Product.DESK_LAMP.label).price).hasText(Product.DESK_LAMP.price);
  }

  @Test
  public void saysSoWhenNothingMatches() {
    productsPage.open();
    productsPage.searchFor("piano");
    assertThat(productsPage.noResults).isVisible();
  }

  @Test
  public void theNotebookCannotBeAdded() {
    productsPage.open();
    assertThat(productsPage.card(Product.NOTEBOOK.label).button).isDisabled();
  }
}
```

src/test/java/com/brainy/shop/tests/cart/AddToCartTest.java

```
package com.brainy.shop.tests.cart;

import com.brainy.shop.base.BaseTest;
import com.brainy.shop.data.Product;
import org.testng.annotations.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@Test(groups = "cart")
public class AddToCartTest extends BaseTest {
  @Test
  public void addsTwoProductsFromTheirCards() {
    signInThroughApi();
    productsPage.open();
    assertThat(productsPage.header.greeting).hasText("Hello, Asha");
    productsPage.card(Product.DESK_LAMP.label).addToCart();
    productsPage.card(Product.COFFEE_MUG.label).addToCart();
    assertThat(productsPage.header.cartCount).hasText("2");

    cartPage.open();
    assertThat(cartPage.items).hasCount(2);
    assertThat(cartPage.total).hasText("₹1,498");
  }
}
```

src/test/java/com/brainy/shop/tests/cart/CheckoutTest.java

```
package com.brainy.shop.tests.cart;

import com.brainy.shop.base.BaseTest;
import com.brainy.shop.data.Product;
import org.testng.annotations.Test;

import java.util.regex.Pattern;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

@Test(groups = {"cart", "smoke"})
public class CheckoutTest extends BaseTest {
  @Test
  public void placesAnOrderForACartFilledThroughTheApi() {
    String token = signInThroughApi();
    api.addToCart(token, Product.WATER_BOTTLE.id, 2);   // arrange through the API, test the UI
    cartPage.open();
    assertThat(cartPage.total).hasText("₹698");

    cartPage.placeOrder();
    assertThat(cartPage.status).hasText(Pattern.compile("Order #\\d+ placed"));
    assertThat(cartPage.header.cartCount).hasText("0");
  }
}
```

Run it: `mvn test -Dsuite=testng-smoke.xml`

Lesson: Step 3: tests by feature, with tags

TABLES

Table: folder · java

Folder	Java
framework code	src/main/java/.../shop
settings	config/Config.java with config.properties
page objects	.../shop/pages/
fixtures	base/BaseTest.java
tests	src/test/java/.../tests/<feature>/*Test.java
runner settings	pom.xml and testng.xml

Lesson: What a framework is, and the finished result

Table: rule · why

Rule	Why
settings come from the environment, with a local default	the same code runs on a laptop, in CI and against staging
secrets never go in the repository	passwords live in CI secrets and in a local .env that Git ignores
exact versions of every package	every machine, and every CI run, uses the same Playwright

Lesson: Step 1: project, settings and runner config

Table: layer · its one job · in this framework

Layer	Its one job	In this framework
data	test users and products	users, products
API helper	fast set up without the browser	ShopApi: signIn, addToCart
components	parts on many pages	Header, ProductCard
page objects	one screen each	BasePage, LoginPage, ProductsPage, CartPage
fixtures	hand objects and a session to each test	test.extend, conftest.py, BaseTest

Lesson: Step 2: data, API helpers, page objects and fixtures

Table: testng

	TestNG
mark a test	@Test(groups = "smoke")
run only those	a suite file with <include name="smoke"/>
leave them out	<exclude name="slow"/>
register the names	not needed

Lesson: Step 3: tests by feature, with tags

Table: when you need · add · covered in

When you need	Add	Covered in
several environments	one .env per environment, or CI variables per job	this chapter, step 1
many signed in roles	a setup step that saves storage state per role	API testing
a run on every pull request	a CI workflow with shards and a merged report	CI/CD
reports for managers and testers	Allure, pytest-html or ExtentReports next to the built-in reports	Reporting labs
visual and accessibility checks	toHaveScreenshot, aria snapshots and axe in their own folder	Visual, accessibility and time
lots of varied test data	small factory functions that build users and orders with unique names	Parallel runs
consistent code	a linter, a formatter and code review rules	your team

Lesson: How the framework grows

TIPS AND TRAPS

IN REAL TEAMS Commit a .env.example with the names of the settings and no values. A new team member copies it to .env and fills it in. The real .env is listed in .gitignore.

Lesson: Step 1: project, settings and runner config

REMEMBER The session fixture signs in through the API and puts the token in the page before it loads. Each test gets its own session, so its cart is its own, and parallel tests cannot disturb each other. This is the isolation rule from the parallel runs chapter, built into the framework.

Lesson: Step 2: data, API helpers, page objects and fixtures

COMMON MISTAKE Java has no fixtures like Playwright Test or pytest. BaseTest plays that part: TestNG runs @BeforeClass, @BeforeMethod and @AfterMethod around every test, and with parallel="classes" each class keeps its own Playwright on its own thread. Playwright objects are not safe to share between threads.

Lesson: Step 2: data, API helpers, page objects and fixtures

IN REAL TEAMS Grow the framework when the tests ask for it. A layer nobody needs yet is code to maintain. The best frameworks are the ones where a new test takes ten lines and needs no new helper.

Lesson: How the framework grows

7.3 CI/CD pipelines

The framework ran with CI settings, a forgotten test.only failed the run, a wrong secret failed the sign in tests, shards merged into one report, and the smoke tests passed inside the official Docker images in all four languages.

KEY IDEAS

- **Every push** Tests run by themselves. Nobody has to remember.
- **Fresh machine** Install everything each time. Or use the Docker image.
- **Secrets** From the CI settings. Masked in the logs.
- **Evidence** Reports kept, pass or fail. Traces explain failures.
- **Remember** CI runs the same three steps everywhere: a machine that can run browsers, the packages and browsers installed, then the tests with CI settings and secrets.

CODE

Run the suite the way the pipeline runs it

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.brainy</groupId>
  <artifactId>shop-tests</artifactId>
  <version>1.0.0</version>

  <properties>
    <maven.compiler.release>21</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <suite>testng.xml</suite>                <!-- mvn test -Dsuite=testng-smoke.xml runs another suite -->
  </properties>

  <dependencies>
    <dependency>
      <groupId>com.microsoft.playwright</groupId>
      <artifactId>playwright</artifactId>
      <version>1.63.0</version>
    </dependency>
    <dependency>
      <groupId>org.testng</groupId>
      <artifactId>testng</artifactId>
      <version>7.11.0</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>3.5.4</version>
        <configuration>
          <suiteXmlFiles>
            <suiteXmlFile>${suite}</suiteXmlFile>
          </suiteXmlFiles>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

Run it: `mvn test && ls target/surefire-reports | grep xml`

Lesson: Running the suite the way the pipeline does

The workflow file each project keeps in .github/workflows

.github/workflows/tests.yml

```
name: Playwright tests
on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    timeout-minutes: 30
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-java@v5
        with:
          distribution: temurin
          java-version: "21"
          cache: maven
      - run: mvn -B test-compile
      - run: mvn -B exec:java -e -Dexec.mainClass=com.microsoft.playwright.CLI -Dexec.classpathScope=test -
Dexec.args="install --with-deps chromium"
```

```
      - run: mvn -B test
        env:
          BASE_URL: ${{ vars.BASE_URL }}
          ASHA_PASSWORD: ${{ secrets.ASHA_PASSWORD }}
          RAVI_PASSWORD: ${{ secrets.RAVI_PASSWORD }}
      - uses: actions/upload-artifact@v5
        if: ${{ !cancelled() }}
        with:
          name: test-results
          path: |
            target/surefire-reports/
            traces/
```

Run it: `git push`

Lesson: GitHub Actions, line by line

What a run looks like when a secret is wrong

src/main/java/com/brainy/shop/data/Users.java

```
package com.brainy.shop.data;

// Test accounts. Real projects read passwords from CI secrets, never from the repository.
public final class Users {
  public static final User ASHA = new User("asha@shop.example", secret("ASHA_PASSWORD", "s3cret"), "Asha");
  public static final User RAVI = new User("ravi@shop.example", secret("RAVI_PASSWORD", "admin123"),
"Ravi");

  private Users() {}

  private static String secret(String name, String fallback) {
    String value = System.getenv(name);
    return value == null || value.isEmpty() ? fallback : value;
  }
}
```

Run it: `ASHA_PASSWORD=wrong mvn test -Dtest="com.brainy.shop.tests.smoke.*Test"`

Lesson: Secrets and settings per environment

Each matrix job runs its own part of the suite

.github/workflows/tests.yml

```
name: Playwright tests (by feature)
on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        feature: [smoke, products, cart]
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-java@v5
        with:
          distribution: temurin
          java-version: "21"
          cache: maven
      - run: mvn -B test-compile
      - run: mvn -B exec:java -e -Dexec.mainClass=com.microsoft.playwright.CLI -Dexec.classpathScope=test -
Dexec.args="install --with-deps chromium"
      - run: mvn -B test -Dtest="com.brainy.shop.tests.${{ matrix.feature }}.*Test"
        env:
          BASE_URL: ${{ vars.BASE_URL }}
          ASHA_PASSWORD: ${{ secrets.ASHA_PASSWORD }}
          RAVI_PASSWORD: ${{ secrets.RAVI_PASSWORD }}
      - uses: actions/upload-artifact@v5
        if: ${{ !cancelled() }}
        with:
```

```
          name: surefire-${{ matrix.feature }}
          path: target/surefire-reports/
```

Run it: `for f in smoke products cart; do echo "== job $f"; mvn test -Dtest="com.brainy.shop.tests.$f.*Test"; done`

Lesson: Shards with a matrix, and one merged report

The framework inside the official Playwright image

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.brainy</groupId>
  <artifactId>shop-tests</artifactId>
  <version>1.0.0</version>

  <properties>
    <maven.compiler.release>21</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <suite>testng.xml</suite>                <!-- mvn test -Dsuite=testng-smoke.xml runs another suite -->
  </properties>

  <dependencies>
    <dependency>
      <groupId>com.microsoft.playwright</groupId>
      <artifactId>playwright</artifactId>
      <version>1.63.0</version>
    </dependency>
    <dependency>
      <groupId>org.testng</groupId>
      <artifactId>testng</artifactId>
      <version>7.11.0</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>3.5.4</version>
        <configuration>
          <suiteXmlFiles>
            <suiteXmlFile>${suite}</suiteXmlFile>
          </suiteXmlFiles>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

Run it: `docker run --rm --init --ipc=host --network host -v "$PWD":/work -w /work -v "$HOME/.m2":/root/.m2 mcr.microsoft.com/playwright/java:v1.63.0-noble mvn -B test -Dsuite=testng-smoke.xml`

Lesson: The official Docker images

A Jenkinsfile for each project

Jenkinsfile

```
pipeline {
  agent { docker { image 'mcr.microsoft.com/playwright/java:v1.63.0-noble' } }
  environment {
    BASE_URL = 'https://staging.shop.example'
    ASHA_PASSWORD = credentials('asha-password')
  }
  stages {
    stage('Test') {
      steps { sh 'mvn -B test' }
    }
  }
  post {
    always {
      junit 'target/surefire-reports/*.xml'
```

```
      archiveArtifacts artifacts: 'traces/**', allowEmptyArchive: true
    }
  }
}
```

Lesson: Jenkins, GitLab and Azure DevOps

TABLES

Table: on your laptop · on ci

	On your laptop	On CI
the machine	yours, with everything already installed	a fresh machine for every run
the browser	installed once	installed in every run, or already in a Docker image
the settings	a local .env file	variables and secrets in the CI settings
a failure	you watch it happen	you read the report, the logs and the trace afterwards
test.only	handy while debugging	must fail the run

Lesson: What CI does for your tests

Table: setting on ci · why

Setting on CI	Why
forbidOnly: true	a forgotten test.only stops the run instead of skipping the rest
retries: 2	a test that fails once and then passes is reported as flaky, not hidden
a fixed number of workers	CI machines are smaller than laptops. The Playwright docs suggest 1 for the most stable runs, or sharding for speed
JUnit XML and HTML reporters	the CI server reads the XML, and people read the HTML
trace: on-first-retry	every retried test leaves a trace to open later

Lesson: Running the suite the way the pipeline does

Table: line · what it does

Line	What it does
on: push / pull_request	when the workflow runs
runs-on: ubuntu-latest	a fresh Linux machine from GitHub
timeout-minutes: 30	stops a job that hangs. Keep it above any global timeout in the test config
actions/checkout	gets the code of the commit
setup-node, setup-python, setup-java	installs the language version
npm ci, pip install -r, mvn	installs the exact package versions from the lock or requirements file
playwright install --with-deps chromium	downloads Chromium and installs the Linux libraries it needs
env: with secrets. and vars.	hands settings and passwords to the tests
if: ${{ !cancelled() }}	uploads the results even when tests failed, which is when you need them most
upload-artifact	keeps the report as a download on the run page

Lesson: GitHub Actions, line by line

Table: trigger · use it for

Trigger	Use it for
push to main	checking what was merged
pull_request	checking a change before it is merged
schedule with cron	a nightly run of the full suite, for example in every browser
workflow_dispatch	a Run workflow button for runs by hand
deployment_status	testing a preview or staging deployment once it is ready

Lesson: GitHub Actions, line by line

Table: kind · in github · in the workflow · seen in logs

Kind	In GitHub	In the workflow	Seen in logs
a setting such as BASE_URL	Settings, Secrets and variables, Actions, Variables	${{ vars.BASE_URL }}	yes
a password	the same page, Secrets	${{ secrets.ASHA_PASSWORD }}	no, GitHub masks it as ***

Lesson: Secrets and settings per environment

Table: maven with testng

	Maven with TestNG
split the suite	one package or suite per job
balance	by hand
one report	one Surefire folder per job

Lesson: Shards with a matrix, and one merged report

Table: language · image

Language	Image
Java	mcr.microsoft.com/playwright/java:v1.63.0-noble

Lesson: The official Docker images

Table: flag · why the playwright docs recommend it

Flag	Why the Playwright docs recommend it
--ipc=host	without it Chromium can run out of memory and crash
--init	avoids special treatment of the process with id 1, a common cause of leftover processes
a fixed tag such as v1.63.0-noble	the image must match the Playwright version in the project. noble means Ubuntu 24.04
--user pwuser with a seccomp profile	only when the tests visit sites you do not trust

Lesson: The official Docker images

Table: ci service · file · sets ci=true · secrets

CI service	File	Sets CI=true	Secrets
GitHub Actions	.github/workflows/*.yml	yes	secrets.NAME
GitLab CI	.gitlab-ci.yml	yes	CI/CD variables, marked masked
Jenkins	Jenkinsfile	no, set it yourself	credentials('id')
Azure Pipelines	azure-pipelines.yml	no, set it yourself	secret pipeline variables, $(NAME)

Lesson: Jenkins, GitLab and Azure DevOps

Table: step · java

Step	Java
1. read the failure in the job log	the Surefire summary
2. download the artifact	target/surefire-reports and traces
3. open the trace	mvn exec:java with show-trace
4. run only the failures again	mvn test -Dtest=CheckoutTest

Lesson: When the pipeline goes red

TIPS AND TRAPS

REMEMBER The Playwright documentation sums CI up in three steps: make sure the machine can run browsers, install the packages and browsers, then run the tests. Everything in this chapter is a version of those three steps.
Lesson: What CI does for your tests

IN REAL TEAMS pytest and TestNG have no test.only, so there is nothing to forbid.
Lesson: Running the suite the way the pipeline does

IN REAL TEAMS In the repository settings, a branch rule can make this workflow a required check. Then a pull request cannot be merged while its tests are red.
Lesson: GitHub Actions, line by line

COMMON MISTAKE Secrets are not given to workflows that run for pull requests from forks, so a fork cannot read your passwords. Tests that need them are skipped or fail there, which is expected.
Lesson: Secrets and settings per environment

REMEMBER fail-fast: false keeps the other shards running when one fails, so you see every failure in one run. The merge job uses if: !cancelled() for the same reason.
Lesson: Shards with a matrix, and one merged report

IN REAL TEAMS In GitHub Actions, the same image goes under `container: image:` in the job, with `options: --user 1001`, and the browser install step is left out.
Lesson: The official Docker images

COMMON MISTAKE Never fix a red pipeline by adding a skip or a longer sleep. Find out whether the app or the test is wrong. The trace from the debugging chapter shows the page at every step of the failed run.
Lesson: When the pipeline goes red

IN REAL TEAMS A test marked flaky in the report passed on a retry. Treat it as a bug to fix this week, not as a pass.
Lesson: When the pipeline goes red

7.4 Reporting labs

One out of date test failed in every lab, and each report showed it: JUnit XML and JSON counts, the Playwright HTML report, pytest-html and ExtentReports with screenshots, Allure Report in all four languages, a custom summary, and one HTML report merged from two shards.

KEY IDEAS

- **For you** list, dot, -v. Right now.
- **For CI** Counts and history. Every server reads it.
- **For testers** Error, screenshot, trace. Why it failed.
- **For everyone** Same report, any language. Trends and labels.

- **Remember** Write several reports from one run: a short one for the terminal, JUnit XML for CI, and an HTML report with evidence for people.

CODE

Lab 1: the reports a machine reads

src/test/java/com/brainy/shop/tests/products/PriceTest.java

```
package com.brainy.shop.tests.products;

import com.brainy.shop.base.BaseTest;
import com.brainy.shop.data.Product;
import org.testng.annotations.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

public class PriceTest extends BaseTest {
  // Out of date on purpose: the lamp now costs ₹1,299, so the reports have a failure to show
  @Test
  public void theDeskLampCosts1199() {
    productsPage.open();
    assertThat(productsPage.card(Product.DESK_LAMP.label).price).hasText("₹1,199");
  }
}
```

Run it: `mvn test; grep -o '<testsuite [^>]*>' target/surefire-reports/TEST-TestSuite.xml`

Lesson: Lab 1: built-in reporters and JUnit XML

Lab 2: an HTML report for people

src/test/java/com/brainy/shop/reporting/ExtentReporter.java

```
package com.brainy.shop.reporting;

import com.aventstack.extentreports.ExtentReports;
import com.aventstack.extentreports.ExtentTest;
import com.aventstack.extentreports.MediaEntityBuilder;
import com.aventstack.extentreports.reporter.ExtentSparkReporter;
import org.testng.IReporter;
import org.testng.ISuite;
import org.testng.ITestResult;
import org.testng.xml.XmlSuite;

import java.util.ArrayList;
import java.util.Base64;
import java.util.Comparator;
import java.util.Date;
import java.util.List;

// TestNG calls generateReport once, after the whole suite. It writes target/extent/index.html
public class ExtentReporter implements IReporter {
  @Override
  public void generateReport(List<XmlSuite> xmlSuites, List<ISuite> suites, String outputDirectory) {
    ExtentSparkReporter spark = new ExtentSparkReporter("target/extent/index.html");
    spark.config().setDocumentTitle("Brainy Shop tests");
    spark.config().setReportName("Brainy Shop, nightly run");
    ExtentReports extent = new ExtentReports();
    extent.attachReporter(spark);
    extent.setSystemInfo("Browser", "Chromium");

    List<ITestResult> all = new ArrayList<>();
    for (ISuite suite : suites) {
      suite.getResults().values().forEach(r -> {
        all.addAll(r.getTestContext().getPassedTests().getAllResults());
        all.addAll(r.getTestContext().getFailedTests().getAllResults());
        all.addAll(r.getTestContext().getSkippedTests().getAllResults());
      });
    }
    all.sort(Comparator.comparingLong(ITestResult::getStartMillis));
    for (ITestResult result : all) add(extent, result);
    extent.flush();
  }
```

```
  private void add(ExtentReports extent, ITestResult result) {
    ExtentTest test = extent.createTest(result.getMethod().getMethodName())
        .assignCategory(result.getTestClass().getRealClass().getSimpleName());
    for (String group : result.getMethod().getGroups()) test.assignCategory(group);
    test.getModel().setStartTime(new Date(result.getStartMillis()));   // the real times, not the report
time
    test.getModel().setEndTime(new Date(result.getEndMillis()));
    switch (result.getStatus()) {
      case ITestResult.SUCCESS -> test.pass("Passed");
      case ITestResult.SKIP -> test.skip("Skipped");
      default -> {
        test.fail(result.getThrowable());
        if (result.getAttribute("screenshot") instanceof byte[] png) {
          String base64 = Base64.getEncoder().encodeToString(png);
          test.fail("Screenshot at failure",
MediaEntityBuilder.createScreenCaptureFromBase64String(base64).build());
        }
      }
    }
  }
}
```

src/test/java/com/brainy/shop/reporting/EvidenceListener.java

```
package com.brainy.shop.reporting;

import com.brainy.shop.base.BaseTest;
import org.testng.ITestListener;
import org.testng.ITestResult;

// TestNG calls onTestFailure while the page is still open, so the screenshot shows the failure
public class EvidenceListener implements ITestListener {
  @Override
  public void onTestFailure(ITestResult result) {
    if (result.getInstance() instanceof BaseTest test) {
      result.setAttribute("screenshot", test.screenshot());   // ExtentReporter reads it at the end
    }
  }
}
```

testng.xml

```
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<!-- Every test, two classes at a time. Each class gets its own browser on its own thread. -->
<suite name="Brainy Shop" parallel="classes" thread-count="2">
  <listeners>
    <listener class-name="com.brainy.shop.reporting.EvidenceListener"/>
    <listener class-name="com.brainy.shop.reporting.ExtentReporter"/>
  </listeners>
  <test name="All tests">
    <packages>
      <package name="com.brainy.shop.tests.*"/>
    </packages>
  </test>
</suite>
```

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.brainy</groupId>
  <artifactId>shop-tests</artifactId>
  <version>1.0.0</version>

  <properties>
    <maven.compiler.release>21</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <suite>testng.xml</suite>                <!-- mvn test -Dsuite=testng-smoke.xml runs another suite -->
  </properties>

  <dependencies>
    <dependency>
      <groupId>com.microsoft.playwright</groupId>
      <artifactId>playwright</artifactId>
      <version>1.63.0</version>
```

```
    </dependency>
    <dependency>
      <groupId>org.testng</groupId>
      <artifactId>testng</artifactId>
      <version>7.11.0</version>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>com.aventstack</groupId>
      <artifactId>extentreports</artifactId>
      <version>5.1.2</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>3.5.4</version>
        <configuration>
          <suiteXmlFiles>
            <suiteXmlFile>${suite}</suiteXmlFile>
          </suiteXmlFiles>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

Run it: `mvn test; ls target/extent`

Lesson: Lab 2: HTML reports with evidence

Lab 3: Allure Report, the same report for every language

src/test/java/com/brainy/shop/tests/products/PriceTest.java

```
package com.brainy.shop.tests.products;

import com.brainy.shop.base.BaseTest;
import com.brainy.shop.data.Product;
import io.qameta.allure.Allure;
import io.qameta.allure.Feature;
import io.qameta.allure.Severity;
import io.qameta.allure.SeverityLevel;
import org.testng.annotations.Test;

import static com.microsoft.playwright.assertions.PlaywrightAssertions.assertThat;

public class PriceTest extends BaseTest {
  // Out of date on purpose: the lamp now costs ₹1,299, so the reports have a failure to show
  @Test
  @Feature("Prices")
  @Severity(SeverityLevel.CRITICAL)
  public void theDeskLampCosts1199() {
    Allure.step("open the products page", () -> productsPage.open());
    Allure.step("check the lamp price",
        () -> assertThat(productsPage.card(Product.DESK_LAMP.label).price).hasText("₹1,199"));
  }
}
```

src/test/java/com/brainy/shop/reporting/EvidenceListener.java

```
package com.brainy.shop.reporting;

import com.brainy.shop.base.BaseTest;
import io.qameta.allure.Allure;
import io.qameta.allure.AttachmentOptions;
import org.testng.ITestListener;
import org.testng.ITestResult;

import java.io.ByteArrayInputStream;

// TestNG calls onTestFailure while the page is still open, so the screenshot shows the failure
```

```java
public class EvidenceListener implements ITestListener {
  @Override
  public void onTestFailure(ITestResult result) {
    if (result.getInstance() instanceof BaseTest test) {
      byte[] png = test.screenshot();
      Allure.attachment("screenshot at failure", "image/png", new ByteArrayInputStream(png),
          AttachmentOptions.withFileExtension(".png"));
    }
  }
}
```

src/test/resources/allure.properties

```
allure.results.directory=target/allure-results
```

pom.xml

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.brainy</groupId>
  <artifactId>shop-tests</artifactId>
  <version>1.0.0</version>

  <properties>
    <maven.compiler.release>21</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <suite>testng.xml</suite>                <!-- mvn test -Dsuite=testng-smoke.xml runs another suite -->
  </properties>

  <dependencies>
    <dependency>
      <groupId>com.microsoft.playwright</groupId>
      <artifactId>playwright</artifactId>
      <version>1.63.0</version>
    </dependency>
    <dependency>
      <groupId>org.testng</groupId>
      <artifactId>testng</artifactId>
      <version>7.11.0</version>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>io.qameta.allure</groupId>
      <artifactId>allure-testng</artifactId>
      <version>3.0.1</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>3.5.4</version>
        <configuration>
          <suiteXmlFiles>
            <suiteXmlFile>${suite}</suiteXmlFile>
          </suiteXmlFiles>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

Run it: `mvn test; npx allure generate target/allure-results && ls allure-report/index.html`

Lesson: Lab 3: Allure Report in all four languages

Lab 4: write your own reporter

src/test/java/com/brainy/shop/reporting/SummaryListener.java

```java
package com.brainy.shop.reporting;

import org.testng.ITestContext;
import org.testng.ITestListener;
```

```java
import org.testng.ITestResult;

// One line per test, then a short summary when the run ends
public class SummaryListener implements ITestListener {
  @Override
  public void onTestSuccess(ITestResult result) { line("PASS", result); }

  @Override
  public void onTestFailure(ITestResult result) { line("FAIL", result); }

  @Override
  public void onTestSkipped(ITestResult result) { line("SKIP", result); }

  @Override
  public void onFinish(ITestContext context) {
    System.out.printf("%d passed, %d failed, %d skipped%n", context.getPassedTests().size(),
        context.getFailedTests().size(), context.getSkippedTests().size());
    context.getFailedTests().getAllResults()
        .forEach(r -> System.out.println("  to look at: " + r.getMethod().getMethodName()));
  }

  private void line(String status, ITestResult r) {
    double seconds = (r.getEndMillis() - r.getStartMillis()) / 1000.0;
    System.out.printf("%s  %s (%.1f s)%n", status, r.getMethod().getMethodName(), seconds);
  }
}
```

testng.xml

```xml
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<!-- Every test, two classes at a time. Each class gets its own browser on its own thread. -->
<suite name="Brainy Shop" parallel="classes" thread-count="2">
  <listeners>
    <listener class-name="com.brainy.shop.reporting.SummaryListener"/>
  </listeners>
  <test name="All tests">
    <packages>
      <package name="com.brainy.shop.tests.*"/>
    </packages>
  </test>
</suite>
```

Run it: `mvn test; echo "exit code $?"`

Lesson: Lab 4: a custom reporter

TABLES

Table: reader · wants · report

Reader	Wants	Report
you, at the terminal	pass or fail, right now	list, line or dot in Playwright Test, -v or -q in pytest, the Surefire summary in Maven
the CI server	counts and history	JUnit XML, which every CI server reads
a tester fixing a failure	the error, the screenshot, the trace	the Playwright HTML report, pytest-html, Allure, ExtentReports
a manager	a dashboard: how many passed, trends, which features	Allure, ExtentReports
your own tools	data in a known shape	JSON, or a custom reporter

Lesson: What a test report is for

Table: evidence · java with testng

Evidence	Java with TestNG
screenshot at failure	page.screenshot() in a listener
video	setRecordVideoDir in the context options
trace	context.tracing() in BaseTest
your own file	Allure.attachment or an Extent media entity

Lesson: Lab 2: HTML reports with evidence

Table: language · adapter · writes results with

Language	Adapter	Writes results with
Java with TestNG	io.qameta.allure:allure-testng (Maven)	nothing: it registers itself as a TestNG listener

Lesson: Lab 3: Allure Report in all four languages

Table: testng

	TestNG
at the start	onStart(context)
after each test	onTestSuccess, onTestFailure, onTestSkipped
at the end	onFinish(context), or IReporter.generateReport
turn it on	a listener in testng.xml

Lesson: Lab 4: a custom reporter

Table: where · how

Where	How
GitHub Actions	upload the report folder as an artifact, or publish it to GitHub Pages
Jenkins	the junit step for counts and history, and the HTML Publisher or Allure plugin for the page
GitLab CI	artifacts: reports: junit, and the report folder as an artifact or GitLab Pages
Azure DevOps	PublishTestResults for JUnit XML, and PublishPipelineArtifact for the HTML

Lesson: Lab 5: merging and publishing reports in CI

Table: need · java

Need	Java
CI counts	Surefire XML, already there
debugging a failure	traces from BaseTest, plus Extent or Allure
one report for many languages and teams	Allure
a report managers know from older Java projects	ExtentReports
something special	a listener

Lesson: Which report should a team use?

TIPS AND TRAPS

REMEMBER

The labs in this chapter all use the framework from the framework chapter, plus one test that fails on purpose: it still expects the lamp to cost ₹1,199 after a price change to ₹1,299. A report is only interesting when something fails.

Lesson: What a test report is for

IN REAL TEAMS The report is only as good as its evidence. A failure with a screenshot and a trace can often be understood in a minute, without running anything again.

Lesson: Lab 2: HTML reports with evidence

NEW IN PLAYWRIGHT This page uses Allure Report 3, which installs from npm and needs Node.js, even for Python and Java projects: `npx allure generate allure-results`. Allure Report 2 is a Java program installed separately, with the command `allure generate`. Both read the same allure-results folder.

Lesson: Lab 3: Allure Report in all four languages

IN REAL TEAMS Keep the allure-results of earlier runs, or the history folder of the last report, and Allure shows trends and flaky tests across runs.

Lesson: Lab 3: Allure Report in all four languages

COMMON MISTAKE A reporter must not change whether the run passed. It only describes the run, so CI still sees exit code 1 when a test failed.

Lesson: Lab 4: a custom reporter

REMEMBER Python and Java shards each write their own JUnit XML file. CI servers read many files at once, for example target/surefire-reports/*.xml in Jenkins, so they do not need merging first. Allure also reads several allure-results folders in one command.

Lesson: Lab 5: merging and publishing reports in CI

IN REAL TEAMS Start with the built-in reports and JUnit XML. Add Allure or Extent when someone outside the test team needs to read the results, not before.

Lesson: Which report should a team use?

7.5 What's new, and AI test agents

A 1.56 feature ran in all four languages, the agent definitions were created for real in the TypeScript framework, and a checklist shows how to keep AI-written tests honest.

KEY IDEAS

- **Releases** New browsers each time. Read the notes first.
- **New APIs** In every language. Like consoleMessages.
- **Test Agents** Plan, generate, heal.
- **Playwright MCP** A browser for assistants. Any language.
- **Remember** Update on a branch and read the release notes. Let AI write tests, but review them like any other code, and make sure each one can fail.

CODE

New in 1.56: read console messages, errors and requests at any time

src/test/java/com/brainy/CollectTest.java

```java
package com.brainy;

import com.microsoft.playwright.ConsoleMessage;
import com.microsoft.playwright.Page;
import com.microsoft.playwright.junit.UsePlaywright;
import com.microsoft.playwright.options.AriaRole;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

@UsePlaywright(ShopOptions.class)
public class CollectTest {
  @Test
  void readsConsoleMessagesAndPageErrorsAfterTheFact(Page page) {
```

```
    page.navigate("/widgets");
    page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Write to the console")).click();
    page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Break the page")).click();
    page.waitForCondition(() -> !page.pageErrors().isEmpty());

    // New in 1.56: the page keeps its recent console messages, errors and requests, no listener needed
    for (ConsoleMessage m : page.consoleMessages()) System.out.println("console." + m.type() + ": " +
m.text());
    for (String e : page.pageErrors()) System.out.println("page error: " + e);
    System.out.println("requests so far: " + page.requests().size());
    assertEquals(1, page.pageErrors().size());
  }
}
```

Run it: `mvn test`

Lesson: A new feature in all four languages

TABLES

Table: version · highlights for testers

Version	Highlights for testers
1.56	Playwright Test Agents: planner, generator and healer. page.consoleMessages(), page.pageErrors() and page.requests()
1.57	a Speedboard tab in the HTML report that shows the slowest tests, and Chrome for Testing builds instead of Chromium
1.58	a timeline chart for merged reports, and old selectors such as _react removed
1.59	page.screencast for videos with chapters and overlays, page.ariaSnapshot(), and npx playwright trace on the command line
1.60	test.abort(), locator.drop() and HAR recording through tracing
1.61	passkeys through browserContext.credentials, page.localStorage, and -G as a short form of --grep-invert
1.62	retryStrategy, WebP screenshots, and npx playwright mcp bundled with Playwright
1.63	test locks, locator.visible(), --add-reporter and a perfetto reporter
1.64	page.webmcp, --shuffle, locator.within() and page.getByRef()

Lesson: How Playwright changes, and how to keep up

Table: language · update the package · then

Language	Update the package	Then
Java	change the version of com.microsoft.playwright in pom.xml	Playwright downloads the matching browsers on first run

Lesson: How Playwright changes, and how to keep up

Table: agent · reads · writes

Agent	Reads	Writes
planner	your running app, starting from the seed test	a Markdown plan in specs/
generator	a plan from specs/	test files in tests/, with each locator and check tried live
healer	a failing test	a fixed test, or a skipped one when it believes the app itself is broken

Lesson: Playwright Test Agents

Table: tool · use it for

Tool	Use it for
Playwright MCP	any assistant that speaks MCP, exploring an app and writing tests in any language
Copy prompt in the HTML report, trace viewer and UI mode	pasting a failed test with its error and page snapshot into any assistant, since 1.51
the Playwright CLI with skills	coding agents. The MCP README notes that the CLI uses fewer tokens

Lesson: Playwright MCP: a browser for AI assistants

Table: check · why

Check	Why
the test uses the framework: page objects, fixtures, test data	otherwise every generated test adds locators that someone must maintain
every check is meaningful	a test that only checks that a heading is visible passes even when the feature is broken
no fixed waits and no force: true	they hide timing problems that real users meet
the healer did not hide a bug	it may skip a test when the app looks broken. A skipped test needs a person to decide
the test fails when the feature breaks	break the feature on purpose once, or change an expected value, and watch it go red
secrets stay out of prompts and plans	plans and prompts are files and messages that other people and tools can read

Lesson: Using AI-written tests safely

TIPS AND TRAPS

IN REAL TEAMS Read the release notes before updating, and update on a branch so CI shows what changed. Each release lists its breaking changes, such as page.accessibility removed in 1.57 and Node.js 16 support removed in 1.54.

Lesson: How Playwright changes, and how to keep up

IN REAL TEAMS Use pageErrors() at the end of a test to fail it when the page threw an error that the user never saw. It is a cheap check that catches broken scripts early.

Lesson: A new feature in all four languages

IN REAL TEAMS Give the agents a seed test that signs in through your own fixtures. Generated tests then start from the same state as the rest of your suite.

Lesson: Using AI-written tests safely